

PIMENTO: A Privacy Framework for Querying Text

Mushtari Sadia* Ang Chen* Amrita Roy Chowdhury*
 *University of Michigan, Ann Arbor

Abstract

Currently, there are two state-of-the-art, complementary privacy guarantees: contextual integrity (CI) for what may flow, and differential privacy (DP) for what may be inferred. Yet neither maps cleanly onto natural language, leaving existing approaches unable to provide these guarantees for analytics over unstructured text. We address this gap with PIMENTO, a framework that takes three forms of natural language: text corpus, queries, and privacy policies; and grounds them into a relational database, creating a common substrate on which both guarantees can be enforced formally. With this design, we not only provide end to end privacy guarantees, but also improvement to utility through three key contributions: *DP aware Text-to-SQL*, which searches for correct queries requiring the least DP noise; *CI aware Text-to-SQL*, which compiles natural language policies into executable CI rules over the database; and a new privacy definition we call *contextual differential privacy*, which redefines the traditional DP neighborhood under CI, and yields a tighter smooth sensitivity bound. Across new benchmarks, PIMENTO selects the best query in 75.3% of cases (upto +45 points over baselines) and achieves zero leakage under correct policy grounding. To our knowledge, PIMENTO is the first framework to provide formal privacy guarantees for natural language analytics under CI, DP, and their composition.

1 Introduction

Organizations increasingly sit on vast amounts of rich information that is trapped in unstructured text – as much as 80% of enterprise data [29]. Hospitals store physician notes, admission reports, medication records, and discharge summaries; law firms maintain case files; companies retain internal communications; public agencies collect similarly sensitive textual data. Across these domains, many important analyses reduce naturally to statistical queries over entities and relationships described in text. Consider a hospital researcher asking: “How many patients reported worsening symptoms

after starting a new HIV medication?” Answering this query requires reasoning and linking evidence scattered across a corpus of documents; the answer cannot simply be read from a single record. Such analyses could be enormously valuable for research, operations, and policy. Yet much of this value remains locked away; the underlying text is highly sensitive, its use is constrained by stringent legal and institutional requirements, such as HIPAA [82] and GDPR [81], and meaningful analysis is only viable with strong privacy guarantees.

Formalizing privacy directly over free-form text, however, is fundamentally difficult. Sensitive information has no fixed representation: it may span sentences, appear through paraphrases, be implied indirectly, or emerge only by combining innocuous facts. There is no obvious “unit” that captures what must be protected. More fundamentally, privacy has two dimensions. First, it is contextual: whether information may be revealed depends on who is asking, what is queried, and under what relationship or policy. Second, even an appropriate aggregate query can reveal information about individuals [21]. No single privacy notion naturally captures both risks. Returning to our hospital example, the query about worsening symptoms may be appropriate for a medical researcher conducting an approved HIV study, but not for a budget analyst unauthorized to access patients’ HIV information. Even the authorized researcher should not receive an exact answer if it could reveal whether a patient is in the queried population.

These risks correspond to two foundational privacy frameworks studied extensively, but independently. Contextual integrity (CI) [6] is normative and governs the appropriateness of an information flow based on the actors, information being queried, and relevant relationships and conditions. In our example, CI may permit the query for a researcher associated with an approved HIV study while denying the same query from a budget analyst. Differential privacy (DP) [21], in contrast, limits information revealed by adding noise calibrated to each individual’s influence on the query answer. Thus, the two guarantees answer different questions about the *same query*: CI determines *what* information may flow; DP determines *how much* that flow reveals.

Unfortunately, existing approaches to enforcing either CI or DP are fundamentally *incompatible* with natural language text. For CI, the formal framework is precise [6]: policies can be expressed in first-order logic over actors, attributes, relationships, and transmission principles. The difficulty is enforcing them over raw text, where these objects are implicit and must be inferred from context. Consequently, prior work has largely relied on LLMs or heuristic contextual classifiers to judge whether information flows are appropriate [26, 43, 58, 76, 80, 87], or on rewriting, minimization, and structured communication layers [1, 5, 26, 60, 73]. These approaches are empirical and provide no end-to-end formal guarantee that the resulting disclosure satisfies the CI policy.

Differential privacy faces a similar barrier, as it requires a well-defined notion of an individual’s contribution. These concepts are natural for structured data such as relational databases, but have no direct analogue in raw text, where information about an individual may be scattered across tokens, sentences, and documents. Existing approaches therefore resort to a weaker relaxation of DP, treating a token or embedding as the protected unit [24, 25, 55, 86], which no longer corresponds to an individual. Even setting this mismatch aside, these mechanisms are impractical: DP in high-dimensional embedding spaces can require enough noise to destroy semantic utility [90], while local-DP sanitization yields coherent text only at privacy parameters too weak for meaningful protection [54]. LLM-based rewriting is also heuristic and provides no formal privacy guarantee [32, 35, 40].

A new privacy substrate for text. Our key insight is to change the representation on which privacy is enforced. Rather than defining CI and DP directly over text, we introduce a structured representation, *relational databases*, between natural language and privacy enforcement. This builds on a broader shift in academia [3, 15, 48, 49] and industry [27, 56, 57, 74, 75, 77, 78], where unstructured text is mapped into a database through *Text-to-DB* [66], and natural-language questions are compiled into executable SQL through *Text-to-SQL* [31, 59, 67, 94]. This paradigm offers the best of both worlds: users retain the flexibility of natural language, while decades of mature database machinery can be brought to bear. The need for structure has also been recognized at the policy level, as recent amendments to CCPA [14, 65] call for bringing “structure to unstructured data” to enable privacy analysis.

Our observation is that this same representation unlocks a qualitatively new benefit: *formal privacy guarantees over text*. Relational databases provide the explicit objects that both CI and DP lack over raw language. For DP, the connection is immediate. Once a natural-language question is translated into SQL, its privacy cost becomes a mathematical property of an explicit query over an explicit database, allowing us to leverage mature DP theory for relational queries [7, 38, 42, 84]. For CI, we make a complementary observation. A CI norm specifies predicates over contextual parameters such as sender, recipient, information type, purpose, and relevant relation-

ships, together with logical conditions governing when a flow is permitted. These predicates can be represented as database relations, while their conjunctions, disjunctions, existential conditions, and negations can be expressed using relational operators and SQL. We can therefore compile a CI norm into an *executable SQL view* whose result is exactly the database tuples satisfying the norm. Unlike asking an LLM whether a disclosure “looks appropriate,” the policy becomes an explicit, mechanically enforceable authorization boundary. The LLM is used only to *translate* natural language into the SQL view, which can be inspected and verified at compile time, rather than making privacy decisions at runtime.

Our contributions. We propose PIMENTO, to the best of our knowledge, the first framework to provide formal privacy guarantees over natural-language text under *either* CI or DP, enabling a new class of privacy-preserving text analytics. PIMENTO is *flexible*: applications can request DP alone, CI alone, or their composition, each formally operationalized.

- **Differential privacy-aware Text-to-SQL.** We formulate a new Text-to-SQL task for finding equivalent lower-sensitivity SQL and develop a new search procedure. We construct the first benchmark for this task, where PIMENTO finds a correct, minimum-sensitivity formulation in **75.3%** of cases, outperforming baselines by up to **45 percentage points**.
- **Contextual integrity-aware Text-to-SQL.** We introduce TEXT-TO-CQL, a new task that grounds natural-language CI norms into SQL views. To the best of our knowledge, this is the first approach to operationalize CI as database views for natural-language analytics. On our new benchmark, current LLMs achieve up to **89% F1**.
- **A formal integration of CI and DP.** We introduce *contextual differential privacy*, a new privacy definition integrating CI and DP by **reshaping the neighborhood relation** of standard DP, and characterize when it yields strictly tighter sensitivity bounds. While prior work has argued for such an integration [8], to the best of our knowledge, we are the first to formalize it as a concrete privacy definition.
- **End-to-end privacy over text.** Together, these components enable a new paradigm for privacy-preserving text analytics: natural language remains the interface, relational databases provide the formal substrate, and privacy enforcement is explicit and auditable rather than delegated to LLM judgments.

2 Background

Contextual Integrity (CI). CI defines privacy by whether information flows appropriately within a particular context. For example, a hospital policy may state that “a doctor may disclose a patient’s diagnosis to a researcher for an approved study,” while prohibiting the same disclosure to a budget analyst. We use the formalization of Barth et al. [6], representing a norm as $N_i = (\lambda_i, r_i^{\text{snd}}, r_i^{\text{rec}}, r_i^{\text{sub}}, \tau_i, \pi_i, \theta_i)$, where $\lambda_i \in \{+, -\}$ indicates whether the flow is permitted or prohibited, $r_i^{\text{snd}}, r_i^{\text{rec}}$,

and r_i^{sub} are the sender, recipient, and subject roles (doctor, researcher, and patient), τ_i is the data type (diagnosis), π_i is the purpose (approved study), and θ_i specifies conditions on the flow. A flow is permitted if it matches a positive norm and no applicable negative norm prohibits it.

Differential Privacy (DP). DP is the de facto standard for protecting individual privacy in aggregate queries.

Definition 1. Let $I \sim I'$ denote databases differing by one tuple in a private relation. A randomized mechanism $\mathcal{M}$ is (ϵ, δ) -differentially private if, for every output set O and $I \sim I'$,

$$\Pr[\mathcal{M}(I) \in O] \leq e^\epsilon \Pr[\mathcal{M}(I') \in O] + \delta.$$

A standard way to achieve DP is to add noise calibrated to query *sensitivity*. For example, the Laplace mechanism releases $q(I) + \eta$, where $\eta \sim \text{Lap}(\Delta q / \epsilon)$ and Δq bounds the change in q between neighboring databases. Global sensitivity, $\text{GS}_q = \sup_{I \sim I'} |q(I) - q(I')|$, takes the worst case over all database instances and can be prohibitively large for queries with joins. In contrast, local sensitivity $\text{LS}_q(I) = \sup_{I' \sim I} |q(I) - q(I')|$ considers only neighbors of the current instance, yielding a tighter bound, but cannot be used directly because its data-dependent noise scale may itself leak information. Smooth sensitivity [61] addresses this with an instance-dependent bound that varies smoothly across neighbors, but is intractable to compute exactly for general multiway joins. We therefore use *residual sensitivity* (RS) [19], an efficiently computable, order-optimal smooth upper bound for this query class, to calibrate noise throughout the paper.

Residual sensitivity (RS). RS is the tightest known efficiently computable smooth sensitivity bound for multiway join counting queries. It captures how a tuple change propagates through joins using data-dependent join multiplicities. Selections can be pushed to their corresponding base relations before computing RS, while projections do not affect the bound. Fig. 1 shows the intuition behind RS. Starting from the full join q , we remove one relation, Z , to form the *residual query* q_E . The *boundary* is where this residual query connects back to the removed relation. Here, the boundary is B . The value b_1 appears three times in q_E , so a tuple added to Z with $B = b_1$ can join with three tuples. This multiplicity of 3 captures how much one tuple change can affect the query answer.

Formally, for a counting query $q = \bowtie_{i=1}^n R_i$, RS considers a residual query q_E which joins a subset of relations $E \subseteq [n]$. Its boundary B_E contains the attributes connecting q_E to the remaining relations. For each private relation R_i , let $E = [n] \setminus \{i\}$. For each boundary value b , $m_i(b)$ is the number of tuples in q_E matching b . RS takes the largest such multiplicity:

$$T_{E_i}(I) = \max_{b \in \text{dom}(B_E)} m_i(b). \quad (1)$$

We denote this maximum by T_{E_i} , the largest number of output tuples one tuple in R_i can affect. Thus, $\text{LS}_i = T_{E_i}$ and $\text{LS}_q(I) = \max_{i \in \mathcal{R}_{\text{priv}}} \text{LS}_i$. Because local sensitivity can vary

$q = X(A, B, C) \bowtie Y(A) \bowtie Z(B, D)$ $q_E = X(A, B, C) \bowtie Y(A)$ $B_E = \{B\}$ (Because only B can join with Z)	$X \bowtie Y$			$X \bowtie Y$ can connect to maximum 3 tuples in Z $m_i(b_1) = 3$ $m_i(b_2) = 1$ $T_{E_i} = 3$
	A	B	C	
	a ₁	b ₁	c ₁	
	a ₁	b ₁	c ₂	
	a ₂	b ₁	c ₃	
	a ₃	b ₂	c ₄	

Figure 1: **RS.** Residual query, boundary, and multiplicity.

sharply across neighbors, RS also bounds how these multiplicities grow with database distance. Let $\mathbf{s} = (s_1, \dots, s_{|\mathcal{R}_{\text{priv}}|})$, where s_i counts tuple changes to private relation R_i , and let $\mathcal{S}_k$ contain all such vectors summing to k . RS computes

$$T_{E, \mathbf{s}}(I) = \sum_{A \subseteq E \cap \mathcal{R}_{\text{priv}}} T_{E \setminus A}(I) \prod_{j \in A} s_j, \quad (2)$$

$$\text{LS}_{q, \mathbf{s}}(I) = \max_{i \in \mathcal{R}_{\text{priv}}} T_{E_i, \mathbf{s}}(I), \quad (3)$$

$$\text{LS}_q^{(k)}(I) = \max_{\mathbf{s} \in \mathcal{S}_k} \text{LS}_{q, \mathbf{s}}(I), \quad (4)$$

$$\text{Finally, } \text{RS}_q(I) = \max_{k \geq 0} e^{-\beta k} \min\{\text{GS}_q, \text{LS}_q^{(k)}(I)\}, \quad (5)$$

where β is the smoothing parameter. We denote the maximizing distance and vector by k^* and $\mathbf{s}^*$.

Key properties. We rely on three properties of RS:

- (P1) **Query dependence.** RS is defined over the complete query; adding a relation can increase or decrease its sensitivity.
- (P2) **Maximizer dependence.** The final RS depends on the maximizing private relation, distance k^* , and distance vector $\mathbf{s}^*$.
- (P3) **Reusable intermediates.** Computing RS produces boundary multiplicities T_E for subsets of relations. We later reuse these in our search procedure.

Text-to-SQL. Recent state-of-the-art tree-based approaches to Text-to-SQL use Monte Carlo Tree Search (MCTS) to explore alternative SQL formulations during generation [44, 89]. MCTS balances *exploitation*, i.e., revisiting promising regions of the search space, with *exploration*, i.e., trying less-explored alternatives. In the tree, a node (v) corresponds to a partial or completed SQL query, and an edge (a) is an LLM reasoning action that produces a child node. Starting from a root node, MCTS tests whether the current node is fully explored (i.e., cannot have more children). If so, a child is chosen using a **selection rule**. Otherwise, an LLM performs **expansion**, proposing a batch of candidate children, from which one is selected randomly. This continues until a terminal node (a complete SQL query) is reached. The query is then evaluated, and a reward $Q(v, a)$ is assigned to the terminal node, typically based on SQL execution agreement. The reward is also backpropagated to every node along that query's branch, while updating the corresponding node and action visit counts $N(v)$ and $N(v, a)$. This constitutes one rollout, and rollouts

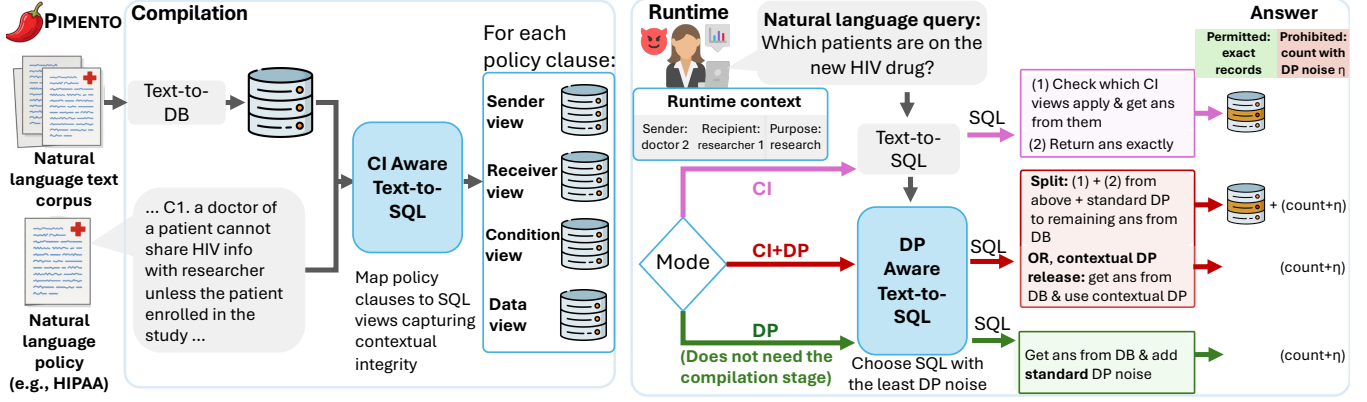


Figure 2: **PIMENTO overview.** PIMENTO takes three natural-language inputs: a text corpus, privacy policy, and query. During policy compilation, it (i) converts the text into a relational database and (ii) compiles the policy into SQL views representing contextual integrity norms using our **CI-aware Text-to-SQL** method. At runtime, given the query and context (sender, recipient, and purpose), PIMENTO supports three privacy-preserving release modes: (i) **CI**, which releases authorized data exactly; (ii) **CI+DP**, which either releases authorized data exactly and unauthorized data as a DP noisy count (*split release*), or returns a noisy count over the full database using **contextual DP**, treating CI-authorized data as public; and (iii) **DP**, which returns a noisy count with standard DP. CI modes use **CI-aware Text-to-SQL**, and DP modes use **DP-aware Text-to-SQL**. Our main contributions are the **CI-aware** and **DP-aware Text-to-SQL** mechanisms and the **definition of contextual DP**; Text-to-DB and standard Text-to-SQL are orthogonal components from prior work.

continue until the budget is exhausted, after which we select a final query with the highest reward.

The selection rule is typically UCT [41]:

$$a^* = \arg \max_a \left[\underbrace{\frac{Q(v, a)}{N(v, a)}}_{\text{exploitation}} + c \underbrace{\sqrt{\frac{\log N(v)}{N(v, a)}}}_{\text{exploration}} \right].$$

Shapley Attribution. A game theoretic method [83] that rewards each decision by its contribution to the outcome.

3 Problem Statement

Motivating Scenario. Our target use case is an organization holding volumes of sensitive unstructured text about individuals. The organization is the data custodian: it controls the corpus, defines its privacy policy, and mediates every query. Analysts may be internal or external, with different roles and access levels. The goal is to answer their analytical questions while enforcing policy and protecting the individuals described in the text.

We use a hospital as our running example. Much of its clinically valuable information is stored in free-form text, including physician notes, admission reports, medication records, and discharge summaries, and queried by diverse stakeholders. Natural language appears in three forms: (i) *organizational data*; (ii) users’ *analytical questions*; and (iii) the *privacy policy* governing access, such as HIPAA. To enable querying with formal privacy enforcement, we retain natural language at the interface but compute over a relational database. Specifically, we assume a Text-to-DB system has transformed the raw corpus into a database D with schema S [3, 36, 66]. We use the **schema below** throughout.

This setting raises three novel tasks, which we define next: (i) *Differential-privacy-aware Text-to-SQL*, generating SQL queries optimized for low DP noise; (ii) *Contextual-integrity-aware Text-to-SQL*, grounding CI policies into the database for enforcement; and (iii) *Integrating CI and DP*, composing both guarantees into end-to-end release mechanisms.

Clinical	Patient(pat_id,...)	Visit(pat_id,diagnosis,...)
	MedicationOrders(pat_id,medication,...)	
	Prescription(pat_id,medication,...)	
	Discharge(pat_id,finaldiagnosis)	
	DxCategory(diagnosis,hivstatus,...)	
Roles	Employee(employee_id,...)	
	TreatingRelationship(employee_id,pat_id,role)	
	Supervises(supervisor_id,supervisee_id)	
	CareTeamMember(employee_id,team_id)	
	TeamAssignment(team_id,pat_id)	
Research	Researcher(researcher_id,...)	Study(study_id,researcher_id)
	StudyPatient(study_id,pat_id)	
Constraint Γ	DxCategory.diagnosis $\rightarrow$ DxCategory.hivstatus	

Table 1: Schema for the running healthcare example. We assume standard primary and foreign key constraints.

3.1 Differential Privacy Aware Text-to-SQL

Since user queries are in natural language, answering them over the relational database requires a Text-to-SQL model that maps the question to a SQL query. Standard Text-to-SQL maps a natural-language question x and schema S to a SQL query $q = \text{T2SQL}(S, x)$, whose answer $q(D)$ is released directly. We consider a different setting: the generated query is executed over a private database and its answer is released under differential privacy,

$$q = \text{T2SQL}(S, x), \quad \hat{a} = q(D) + \eta, \quad \eta \sim \text{Lap}\left(\frac{\text{RS}_q(D)}{\epsilon}\right).$$

This seemingly simple change introduces two challenges. First, query generation becomes part of the privacy mechanism: if the choice of q depends on the private database, it must itself satisfy DP. Second, utility depends on the chosen query’s sensitivity $RS_q(D)$, which determines the noise added to its answer. Existing DP query-processing techniques sidestep the first challenge by assuming q is fixed and address the second *post-hoc*, by reducing its sensitivity or release noise [7, 19, 38, 42, 84]. Text-to-SQL, however, presents a new opportunity: the query itself is *not* fixed.

Ambiguity as Opportunity. We turn an inherent challenge of Text-to-SQL, ambiguity, into an opportunity for improving privacy utility. While ambiguity about the user’s intent is a flaw, having multiple correct ways to express the same intent is a feature. A natural-language question can admit multiple semantically correct SQL formulations because schemas often represent the same relationship through different tables, foreign keys, or join paths [9, 16, 17, 62]. Conventional Text-to-SQL has little reason to prefer among correct formulations. Under DP, however, their differences can yield substantially different sensitivities, and thus different noise.

Consider the query “How many patients were prescribed drug X by cardiologists?” The database supports two correct paths: `Prescription` $\bowtie$ `Employee` and `MedicationOrder` $\bowtie$ `Visit` $\bowtie$ `Employee`. Both return the same answer, but can have very different sensitivities: the first may associate one cardiologist with hundreds of prescriptions, whereas the second may associate each visit with only a few medication orders. Thus, different join paths induce different multiplicities and, consequently, different sensitivity. Note that choosing the lowest-sensitivity formulation is **database-dependent**: neither path here is inherently better, and the question and schema alone cannot determine which has lower sensitivity.

These observations motivate a new problem that we call **DP-aware Text-to-SQL**: given a natural-language question x , schema S , and private database D , the goal is to select, among semantically correct SQL formulations, a query with low sensitivity while ensuring that the selection itself preserves differential privacy. In other words, we ask:

Can Text-to-SQL privately select the correct query that is cheapest to release under DP?

3.2 Contextual Integrity Aware Text-to-SQL

For CI, we are given a natural-language policy $\mathcal{P}$, the database constructed from the raw text, and a user’s query translated into SQL. The goal is to answer the query while respecting the information flows permitted by $\mathcal{P}$. Prior work commonly hands CI norms to an LLM, and asks it to decide what may be revealed. This is fundamentally limited. First, authorization remains a model judgment with no formal guarantee [5, 72]. Second, LLMs make substantial contextual privacy errors: ConfAId reports private information disclosure in 57% of

cases [58]. Third, placing the privacy decision inside the LLM exposes enforcement to prompt-injection attacks [5, 92]. Our key observation is that CI norms are logical rules and should therefore define the authorization boundary, rather than merely guide an LLM. We use the LLM only to *translate* natural-language norms into executable rules over the schema; the final privacy decision is deterministic.

Shared Representation as Opportunity. To make this translation possible, we make a key observation: CI norms and SQL share a common logical foundation. A CI norm specifies who may access what information under what conditions, which can be expressed as predicates over database relations and attributes. Thus, once grounded in the schema, a CI norm can be compiled directly into executable SQL.

Proposed Workflow. Consider a hospital policy: an attending physician may disclose a patient’s diagnosis to a resident for treatment, only if the attending treats the patient and supervises the resident. We can compile this clause into a CI norm, represented by SQL views: a sender view identifies attendings from `TreatingRelationship`, a recipient view identifies residents from `Employee`, a data view locates diagnosis information in `Visit`, and condition views encode the required treatment and supervision relationships using join queries between these tables. Hereafter, a *CI norm* refers to this collection of SQL views representing its roles, data, purpose, and conditions.

At runtime, these views determine which norms apply and the data they authorize. Suppose resident 20 queries diagnosis data from physician 10. We can identify norms whose data views cover the queried information, bind physician 10 and resident 20 to their views, and execute them over the database. The result contains only diagnoses satisfying the norm’s conditions. Repeating this for all applicable norms, and subtracting prohibited data from permitted data, enables releasing the resulting authorized data exactly.

This motivates the problem we call **contextual integrity aware Text to SQL**, or **TEXT-TO-CQL** in short.

TEXT-TO-CQL: Given a natural-language privacy policy $\mathcal{P}$, relational schema S and a user query, how can we compile the policy into executable SQL views that, instantiated with the query context at runtime, enforce exactly the information the query is authorized to release under contextual integrity?

3.3 Integrating Contextual Integrity and Differential Privacy

So far, we have defined tasks for enforcing CI and DP independently in our setting. Yet integrating the two can provide benefits that neither offers alone. Contextual integrity is *normative*: it determines whether an information flow is appropriate. Differential privacy is *descriptive*: it limits what can be learned about individuals from that flow through calibrated noise, both *what* information may flow and *how much* that flow may reveal. There has been growing interest in bringing these two

perspectives together. Benthall and Cummings [8] explicitly advocate integrating CI and DP, while related work on policy-aware DP shows that public or policy-specified information can be incorporated into sensitivity analysis [11, 30, 39]. What has been missing is an operational bridge: CI norms have traditionally remained logical specifications rather than executable objects that a DP mechanism can directly consume. PIMENTO provides this bridge. By compiling natural-language CI norms into executable SQL views (Sec. 3.2), we obtain a concrete, database-grounded authorization boundary that can be composed directly with DP.

Beyond making CI and DP independently composable within our end to end setting, we identify a further opportunity from their integration: improving DP utility. Standard DP protects all records uniformly, even when CI already authorizes some information to flow to the querier. But if that information is already appropriate to reveal, should it still be used for noise calibration? Our key insight here is to **revisit the notion of DP neighborhood** under contextual integrity. Since we can identify the records authorized by CI at runtime, we can treat those records as public and hold them fixed in the neighborhood relation. This can reduce local sensitivity by excluding changes to already authorized information. The remaining question is whether this reduction also yields a tighter smooth upper bound through residual sensitivity.

This leads to our third problem:

Can contextual integrity enable tighter smooth sensitivity bounds for differentially private query answering?

4 PIMENTO: System Description

PIMENTO is a privacy framework for natural-language analytics over unstructured text. To the best of our knowledge, PIMENTO is the first framework to provide formal privacy guarantees for natural-language text under either CI or DP, and to support their combination within a unified framework.

As motivated in Sec. 3, we envision PIMENTO being deployed by the data custodian, e.g., the hospital. It takes three natural-language inputs: (i) the text corpus, (ii) a user query, and (iii) a privacy policy, and enforces the requested privacy semantics before releasing an answer. It is *flexible*: applications may request DP alone, CI alone, or their composition, and PIMENTO formally operationalizes each choice.

PIMENTO realizes these guarantees by grounding all three inputs in a common relational database representation. The text corpus is converted into a relational database using existing Text-to-Database techniques, while the user’s question is translated into SQL using Text-to-SQL. Our DP modes support single and multiway join counting queries with selections and projections, while CI alone supports arbitrary SQL queries. Database construction is orthogonal to PIMENTO: it is a well-studied problem, and PIMENTO can build on any existing Text-to-Database system [3, 36, 48, 49, 66]. Recent work

already achieves over 95% performance on this task [66], so we focus on the privacy layer over the resulting representation. We develop DP-aware Text-to-SQL to optimize query generation for private release, CI-aware Text-to-SQL to compile natural-language CI policies into enforceable database views, and mechanisms for integrating the resulting authorization boundary with DP. We describe each component next, followed by guidelines for choosing among these guarantees. Fig. 2 provides an overview.

4.1 Differential Privacy Aware Text-to-SQL

We start with two strawman solutions to build intuition.

- **Strawman Solution 1.** A natural approach is to fine-tune models to favor low-sensitivity queries. However, no SQL is universally low-sensitivity: residual sensitivity is *data-dependent*. Thus, a model trained on one set of databases cannot reliably identify the lowest-sensitivity formulation on a new database, as our experiments confirm (Sec. 5.1).
- **Strawman Solution 2.** Another approach is to generate the top- k candidates, privately evaluate their sensitivities, and select the minimum. This has two limitations. First, evaluating more candidates consumes privacy budget because residual sensitivity is data-dependent. Second, the lowest-sensitivity formulation may lie outside the top- k , using alternative relations, join paths, or predicates that receive low model probability. Our experiments confirm that top- k candidates often lack this structural diversity (Sec. 5.1). At the other extreme, exhaustive search covering all structures is intractable.

These limitations require sensitivity optimization at *runtime*, using database-dependent feedback to guide search. This in turn creates two challenges: (1) **Use of private data-dependent feedback**, since the search uses private database-dependent feedback, and (2) **Correctness-preserving sensitivity optimization**, since reducing sensitivity must not come at the cost of semantic correctness. Tree search is a natural fit: unlike top- k decoding, MCTS uses feedback from explored queries to search beyond the model’s initial distribution. PIMENTO extends MCTS by making *sensitivity a first-class search objective*, steering search toward semantically correct, low-sensitivity formulations while preserving privacy. We realize this in four stages, illustrated in Fig. 3.

4.1.1 Initialization

At initialization, PIMENTO uses three techniques:

Private Synthetic Dataset. MCTS evaluates many candidates using two data-dependent signals: (i) residual sensitivity for DP utility and (ii) result self-consistency for semantic correctness. Repeatedly computing these on D would accumulate privacy loss. Instead, PIMENTO pays for data dependence once by generating a private synthetic database $\tilde{D} \leftarrow \mathcal{M}_{\text{syn}}(D; \epsilon_{\text{syn}}, \delta_{\text{syn}})$ before search and using $\tilde{D}$ for all

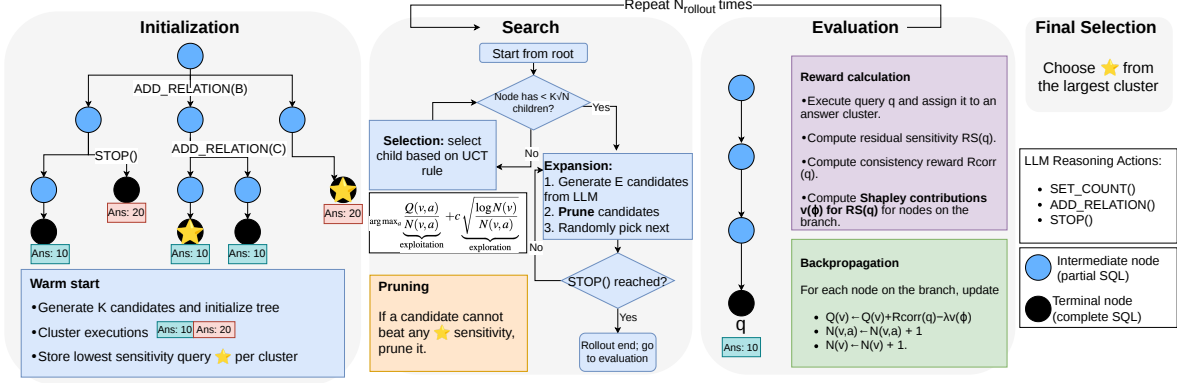


Figure 3: The four-stage Monte Carlo Tree Search procedure for **DP-aware Text-to-SQL** in PIMENTO.

computations. This incurs $(\epsilon_{\text{syn}}, \delta_{\text{syn}})$ once; all subsequent use of $\tilde{D}$ is post-processing. We use PrivPetal [10], which preserves cross-table correlations in relational data.

DP-Compatible Search Tree. DP also constrains what MCTS may generate. Generic Text-to-SQL can introduce arbitrary SQL operators, while our DP mechanism supports counting queries over multiway joins with selections and projections (Sec. 2). We therefore make the search DP-compatible by construction: each node is a partial query, and each edge is restricted to one of three actions:

- **SET_COUNT(x)** sets the aggregation to $\gamma_{\text{COUNT}(x)}$, where x is $*$ ¹ or an attribute, with $*$ as the default.
- **ADD_RELATION(R, J, F)** adds relation R with join predicate J and filter F . For the first relation, $E \leftarrow \sigma_F(R)$; otherwise, $E \leftarrow \sigma_F(E \bowtie_J R)$.
- **STOP()** terminates construction and returns $q = \gamma_{\text{COUNT}(x)}(E)$.

Warm-start. Since zero shot Text-to-SQL is optimized for accuracy, we use K zero shot samples to warm start the tree and simplify the problem, decomposing each sample into our actions and inserting it as a tree path. We execute these queries on $\tilde{D}$ and group those with the same answer into *execution clusters* C_y . Queries within a cluster agree on their answer but may differ structurally and in sensitivity. For each cluster, we maintain its lowest-sensitivity *incumbent* q_y . Clusters and incumbents are updated as the search progresses.

4.1.2 Search

As described in Sec. 2, MCTS alternates between selection and expansion to explore new query formulations. However, broad exploration for low-sensitivity formulations is computationally expensive, while naively expanding every branch quickly becomes intractable. PIMENTO therefore uses two complementary techniques to better focus the search:

Sensitivity-Guided Pruning. PIMENTO uses a greedy pruning

strategy to eliminate branches whose next action is unlikely to improve any formulation found so far. Consider a partial query q_v and a proposed equijoin to relation R on $q_v.A = R.B$. We consider the extension only when it is *containment preserving* on $\tilde{D}$, meaning every join value in q_v has a match in R : $\pi_A(q_v(\tilde{D})) \subseteq \pi_B(R(\tilde{D}))$. Although sensitivity can generally increase or decrease as relations are added (**P1, Sec. 2**), containment preservation identifies a special case: if R retains every join value from q_v , adding R cannot lower sensitivity because it preserves the maximum join multiplicity. This is a greedy condition, as the guarantee need not hold after subsequent actions. Let $q_{v'}$ denote the extended query (with the proposed action) and $\iota_{\max} = \max_y \iota_y$ the largest sensitivity incumbent among the execution clusters found so far. We prune when $\text{RS}_{q_{v'}}(\tilde{D}) > \iota_{\max}$. The proposed action has already raised sensitivity above every known cluster incumbent, making the branch less promising. We delay pruning for a few rollouts to allow the clusters and incumbents to stabilize.

Disallowing Duplicate Branches. Sibling nodes using the same relation key create redundant subtrees. We pass existing sibling keys as an exclusion set when generating children and discard any that reuse them.

4.1.3 Evaluation

When a rollout reaches **STOP()**, it produces a completed query q , which PIMENTO evaluates for correctness and sensitivity. We design two rewards:

Correctness Reward. We estimate correctness using result self-consistency. After observing M valid queries, we define

$$R_{\text{corr}}(q) = \frac{|C_y(q)|-1}{M-1}, \quad R_{\text{corr}}(q) = 0 \text{ if } M \leq 1. \quad (6)$$

Thus, queries supported by larger execution clusters receive higher reward.

Sensitivity Reward. By (**P1**), RS is defined only for a *complete* query, whereas MCTS constructs queries one relation at a time. Scoring partial queries is insufficient because adding a relation can raise or lower sensitivity. A terminal reward such

¹COUNT($*$) returns the total number of result rows.

as $1/(1 + \log \text{RS}_q(\tilde{D}))$ is also too coarse: it assigns every action the same signal despite their potentially different effects. We also confirm that this performs poorly (Sec. 5.1).

We instead treat sensitivity feedback as a relation-level credit-assignment problem. Viewing the relations of q as players in a cooperative game with value $\text{RS}_q(\tilde{D})$, we use Shapley attribution [83] to assign each relation a contribution ϕ_i . These contributions sum to $\text{RS}_q(\tilde{D})$ and are independent of construction order. We (i) define a value function over relation subsets using quantities already computed by RS, (ii) compute each relation’s Shapley contribution, and (iii) normalize and negate these contributions so sensitivity-increasing relations are penalized and sensitivity-reducing ones rewarded. This converts query-level RS into an action-level signal for MCTS.

Sensitivity Value Function. Consider a completed query $q = \bowtie_{i=1}^n R_i$ with relation indices $V = [n]$. For distance vector $\mathbf{s}$,

$$\text{RS}_q(\tilde{D}) = \max_{k \geq 0} e^{-\beta k} \min \left\{ \text{GS}_q, \max_{\mathbf{s} \in \mathcal{S}_k} \max_{i \in \mathcal{R}_{\text{priv}}} T_{V \setminus \{i\}, \mathbf{s}}(\tilde{D}) \right\}.$$

(Sec. 2). For each residual query q_E , RS computes

$$T_{E, \mathbf{s}}(\tilde{D}) = \sum_{A \subseteq E \cap \mathcal{R}_{\text{priv}}} T_{E \setminus A}(\tilde{D}) \prod_{j \in A} s_j.$$

Thus, terminal RS already computes maximum boundary multiplicities T_E for relation subsets (P3). PIMENTO caches and reuses them to attribute to individual relation choices.

By (P2), the terminal RS depends on its maximizing distance k^* and distance vector $\mathbf{s}^*$. We therefore retain these maximizing values from the terminal computation and, holding them fixed, define for each $U \subseteq V$

$$W(U) = e^{-\beta k^*} \min \left\{ \text{GS}_q, \max_{i \in U \cap \mathcal{R}_{\text{priv}}} T_{U \setminus \{i\}, \mathbf{s}^*}(\tilde{D}) \right\}. \quad (7)$$

where the maximum over an empty set is defined as zero. Hence $W(\emptyset) = 0$ and, because the terminal maximizing values are retained, $W(V) = \text{RS}_q(\tilde{D})$.

Importantly, $W(U)$ is not the residual sensitivity of the subquery containing only U . Instead, it measures the contribution of relations in U to the fixed terminal residual-sensitivity computation: we retain k^* , $\mathbf{s}^*$, and GS_q rather than re-optimizing residual sensitivity for every subset. We refer to $\{W(U) : U \subseteq V\}$ as the sensitivity vector.

Shapley attribution. With the value function W , we divide the total sensitivity $W(V)$ among the relations. A relation can affect residual sensitivity by changing either the maximum boundary multiplicities, or which private relation attains the maximum in Eq. 7. Shapley attribution captures both *independently of the order* in which MCTS constructs the query, so each relation’s contribution reflects its intrinsic effect rather than the search trajectory. For relation R_i , its Shapley contribution is

$$\phi_i = \sum_{U \subseteq V \setminus \{i\}} \frac{|U|!(n-|U|-1)!}{n!} [W(U \cup \{i\}) - W(U)]. \quad (8)$$

Here U is a subset of relations that could already be present before R_i is added, and $W(U \cup \{i\}) - W(U)$ measures how much

the terminal computation changes when R_i joins U . Because this effect depends on which relations are already present, we average it over every $U \subseteq V \setminus \{i\}$. The weight $\frac{|U|!(n-|U|-1)!}{n!}$ is the fraction of the $n!$ relation orderings in which exactly U precedes R_i ($|U|!$ orders before, $(n-|U|-1)!$ after), so ϕ_i is R_i ’s order-independent contribution to residual sensitivity.

Since $W(\emptyset) = 0$, $W(V) = \text{RS}_q(\tilde{D})$, Shapley efficiency gives

$$\sum_{i \in V} \phi_i = W(V) - W(\emptyset) = \text{RS}_q(\tilde{D}).$$

Thus, the Shapley values exactly decompose the residual sensitivity of the full query into relation-level contributions.

Example. Consider $q = x \bowtie y \bowtie z$, where all three relations are private. If a tuple in x changes, the remaining relations $\{y, z\}$ determine how much that change can be amplified. At the distance vector which resulted in the maximum residual sensitivity of the final query $\mathbf{s}^*$, this bound is

$$T_{\{y, z\}, \mathbf{s}^*}(\tilde{D}) = T_{\{y, z\}}(\tilde{D}) + s_y^* T_{\{z\}}(\tilde{D}) + s_z^* T_{\{y\}}(\tilde{D}) + s_y^* s_z^* T_{\emptyset}(\tilde{D}).$$

Changing a tuple in y or z analogously gives $T_{\{x, z\}, \mathbf{s}^*}(\tilde{D})$ and $T_{\{x, y\}, \mathbf{s}^*}(\tilde{D})$, respectively. The maximum over these cases determines the local-sensitivity term used by $W(\{x, y, z\})$.

The Shapley contribution of z then averages its marginal effect across all possible contexts:

$$\begin{aligned} \phi_z = & \frac{1}{3} [W(\{z\}) - W(\emptyset)] + \frac{1}{6} [W(\{x, z\}) - W(\{x\})] \\ & + \frac{1}{6} [W(\{y, z\}) - W(\{y\})] + \frac{1}{3} [W(\{x, y, z\}) - W(\{x, y\})]. \end{aligned}$$

For example, $W(\{x, z\}) - W(\{x\})$ measures the effect of adding z when x is already present. The coefficients account for how often each term occurs across all possible relation orderings. Thus, $\phi_z < 0$ means that adding z lowers sensitivity on average, whereas $\phi_z > 0$ means that it raises sensitivity.

Sensitivity credit. We normalize each Shapley contribution as $v(x) = \text{sgn}(x) \log(1 + |x|) / \log(1 + S_{\text{ref}})$, where S_{ref} is the median warm-start RS, compressing its magnitude while preserving its sign. If action a_i adds relation R_i , it receives

$$R_{\text{sens}}(i, q) = -\lambda_s v(\phi_i). \quad (9)$$

where λ_s controls its weight.

Backpropagation. The two rewards are backpropagated at different granularities. Correctness is a query-level signal and is propagated along the entire trajectory. Sensitivity is relation-specific and is assigned only to the edge that introduced the corresponding relation. Thus, for an action a_i we update:

$$\begin{aligned} Q(v_{t-1}, a_i) & \leftarrow Q(v_{t-1}, a_i) + R_{\text{corr}}(q) - \lambda_s v(\phi_i), \\ N(v_{t-1}, a_i) & \leftarrow N(v_{t-1}, a_i) + 1, \\ N(v_{t-1}) & \leftarrow N(v_{t-1}) + 1. \end{aligned} \quad (10)$$

For actions that do not add a relation, only $R_{\text{corr}}()$ is applied. Thus, every action receives credit for correctness, while relation-adding actions also get credit for sensitivity reduction, letting MCTS learn which choices yield lower sensitivity.

4.1.4 Final Selection

At the end of search, we select the lowest sensitivity incumbent query from the largest cluster $y^* = \arg \max_y |C_y|$, $q^* = q_{y^*}$. We then compute $RS_{q^*}(D)$ on the private database to calibrate noise and release the answer. Thus, by choosing the largest cluster first, correctness takes priority over low sensitivity. Algorithm 1 in App. A.3 summarizes the complete search.

Theorem 1 (DP of Text-to-SQL). *If the synthetic release is $(\epsilon_{\text{syn}}, \delta_{\text{syn}})$ -DP and the final query release is $(\epsilon_{\text{rel}}, \delta_{\text{rel}})$ -DP, then PIMENTO is $(\epsilon_{\text{syn}} + \epsilon_{\text{rel}}, \delta_{\text{syn}} + \delta_{\text{rel}})$ -DP (proof in App. A.2).*

4.2 Contextual Integrity Aware Text-to-SQL

PIMENTO takes a natural-language privacy policy $\mathcal{P}$ and database schema $\mathcal{S}$ and compiles the policy into SQL views encoding what information may be disclosed, to whom, and under what conditions. We call this task TEXT-TO-CQL. At runtime, PIMENTO instantiates these views with the query context to derive V_p^{exact} , the exact information authorized for disclosure. This design removes the LLM from the privacy decision: the LLM only translates policy text into SQL, while authorization is enforced symbolically over the compiled views. This avoids relying on LLM judgment for privacy decisions and enables systematic validation of its output, including detecting compilation errors when a policy clause cannot be represented in the database.

This introduces three challenges: **(1) Grounding symbolic norms**, mapping policy concepts such as roles, information, and conditions to concrete database operations; **(2) Resolving interactions across norms**, ensuring every policy clause is represented (coverage), norms do not conflict or duplicate one another (satisfiability and non-redundancy), and permitted information does not imply prohibited information (information implication); and **(3) Enforcing context at runtime**, determining authorization for each query based on its context and database state. PIMENTO addresses these through three steps: *policy-to-view translation* compiles CI norms into SQL views, *view canonicalization* validates and resolves interactions among them, and *runtime enforcement* instantiates them to derive the authorized information $\mathcal{A}_c(D)$. The first two run once per policy and schema, while enforcement runs per query. We consider the following running policy:

An attending physician may disclose a patient’s diagnosis to a resident for treatment if the resident is supervised by the attending and is on the patient’s care team.

4.2.1 Policy-to-View Translation.

The core of TEXT-TO-CQL is translating each natural-language policy clause into executable database views. PIMENTO first extracts a structured CI norm N_i from each

policy clause, following prior work [23, 45, 70], and then grounds each component in $\mathcal{S}$ using an LLM. We provide the norm, source clause, schema, and examples for three types of views. *Role views* ($V_i^{\text{snd}}, V_i^{\text{rec}}, V_i^{\text{sub}}$) identify valid sender, recipient, and subject bindings; in our example, they map the attending and resident roles to Employee. *Condition views* (V_i^θ) encode the relationships required by the policy. These may require a lookup, such as the attending relationship in TreatingRelationship; a join, such as care-team membership through CareTeamMember and TeamAssignment; or recursion, such as indirect supervision through Supervises. Finally, *data views* (V_i^{data}) identify all locations containing the governed information. For example, Diagnosis may appear in both DxCategory and Discharge; PIMENTO unions these locations and tags each value with its provenance $o = (R, \text{tid}, A)$, identifying the source relation, tuple, and attribute. This provenance is later used for the neighborhood relation in Sec. 4.3. Therefore, for each N_i , translation produces $\mathcal{V}_i = \{V_i^{\text{snd}}, V_i^{\text{rec}}, V_i^{\text{sub}}, V_i^\theta, V_i^{\text{data}}\}$, which jointly captures the CI constraints.

4.2.2 View Canonicalization and Verification

Compiling each norm independently can violate *coverage*, *satisfiability*, and *non-redundancy*, while missing *information implications* across norms. Our key insight is that grounding the norms in the database enables symbolic checks for these violations. We perform four checks over the compiled views.

(1) Coverage: we assign each policy clause an identifier and ensure that it produces at least one compiled view. If not, PIMENTO reports the clause as uncompileable and returns a policy compilation error. **(2) Redundancy:** PIMENTO executes the compiled views over applicable database records and compares their outputs. Views governing the same information are merged, yielding a canonical policy representation. **(3) Satisfiability:** individually valid norms may collectively produce contradictory permissions and prohibitions. PIMENTO therefore evaluates them together and checks that the policy admits at least one valid information flow; otherwise, it reports the policy as unsatisfiable. **(4) Information implication:** PIMENTO identifies when information governed by one view reveals information governed by another and materializes this dependency into the policy. For example, if $\text{Diagnosis} \rightarrow \text{HIVStatus}$ and HIV status is prohibited, PIMENTO adds a prohibition over diagnosis records, ensuring they are also excluded at runtime. Thus, the result is a consistent set of policy views ready for runtime enforcement.

4.2.3 Runtime Policy Enforcement

The compiled views still contain unknown parameters (e.g., who exactly is the querier?), and must be contextualized for each query. We therefore require each query to be also supplied with the runtime context: the sender s_p , querier r_p ,

declared purpose π_p , and SQL query q_p , representing a query as $p = (s_p, r_p, \pi_p, q_p)$. Enforcement proceeds in three steps.

(1) Identify relevant norms. PIMENTO first identifies the attributes returned by q_p . For example, if the query queries patient IDs and diagnoses from Visit, the returned attributes are Visit.pat_id and Visit.Diagnosis. It then selects norms whose data views V_i^{data} govern any of these attributes, restricting enforcement to relevant norms.

(2) Instantiate and evaluate norms. For each relevant norm, PIMENTO substitutes the query context (s_p, r_p, π_p) into its views and evaluates them over the current database. Suppose attending physician 10 queries diagnosis information for resident 20 for treatment. PIMENTO binds physician 10 as the sender and resident 20 as the recipient, then evaluates the role and condition views to identify patients for whom the policy conditions hold. The data view then identifies the diagnosis values governed by the norm. Repeating this for all relevant positive and negative norms produces the permitted information Pos_p and prohibited information Viol_p , giving the final authorized set $V_p^{\text{exact}} = \text{Pos}_p \setminus \text{Viol}_p$.

(3) Construct the authorization boundary. V_p^{exact} gives exactly the data authorized for release and can directly answer the query under CI. For integration with DP in the next section, we also identify the corresponding database cells. Since each data view retains the origin $o = (R, \text{tid}, A)$ of every value, PIMENTO maps V_p^{exact} back to its source cells to form $\mathcal{A}_c(D)$.

Theorem 2 (Equivalence to contextual integrity). *Assuming the CI norms are faithfully compiled by the LLM, the resulting SQL views are equivalent to contextual integrity: for any database and query, they authorize exactly the information flows permitted by CI (proof is in App. A.2).*

4.3 Integrating Contextual Integrity and Differential Privacy

The semantics of integrating CI and DP is as follows. We wish to release a query's answer under DP, but we now have additional contextual information about the query: the CI policy specifies which information flows are authorized and any record authorized to flow is effectively public to the querier. Intuitively, this should reduce the sensitivity that DP must protect against. The key lies in the neighborhood: standard DP protects against all neighboring databases; CI rules out neighbors that differ only in information already authorized to the querier. This yields a finer-grained neighborhood that holds contextually public information fixed while continuing to protect what remains private. Unlike prior approaches that assume coarse public information such as relation cardinalities [11], CI can expose richer structure, including individual tuples, attributes, and relationships. We formalize this as *contextual differential privacy* and characterize exactly when it yields tighter sensitivity bounds than standard DP.

Definition 2 (Contextual Differential Privacy). *Fix a query context $c = (s_p, r_p, \pi_p, q_p)$. Let $\mathcal{A}_c(D)$ denote the information in D that the CI policy authorizes to flow under context c . Define the contextual neighboring relation*

$$D \sim_c D' \iff D \sim D' \wedge \mathcal{A}_c(D) = \mathcal{A}_c(D'),$$

where $D \sim D'$ denotes the standard neighboring relation.

A randomized mechanism $\mathcal{M}$ satisfies (ϵ, δ) -contextual differential privacy for context c if, for all $D \sim_c D'$ and all measurable sets of outputs O ,

$$\Pr[\mathcal{M}(D, c) \in O] \leq e^\epsilon \Pr[\mathcal{M}(D', c) \in O] + \delta.$$

Example. Suppose a resident queries diagnosis data for treatment. The database contains patients P1, P2, and P3, whose removal changes the query answer by 10, 6, and 2, respectively. Under add/remove DP, the worst-case change is 10. Now suppose CI authorizes the diagnoses of P1 and P2, placing them in $\mathcal{A}_c(D)$. Contextual neighborhood must preserve $\mathcal{A}_c(D)$, leaving only P3 as a valid removal and reducing the worst-case change to 2. However, insertion of a new unauthorized record remains possible: if a new P4 changes the answer by 12, the insertion still preserves $\mathcal{A}_c(D)$. Thus, CI may restrict removals without restricting insertions, unless schema constraints limit insertions. These refinements can tighten residual sensitivity.

Next, we characterize when CI tightens local sensitivity (Eq. 4). Recall P2: since RS is determined by the maximizing private relation, these reductions matter only when they affect that relation.

Rule language. For a relation R_i , we consider the following grammar of public information²:

$$P ::= \perp \mid R_i \mid \sigma_\phi(R_i) \mid \pi_{B_i}(R_i) \mid \pi_{B_i}(\sigma_\phi(R_i)).$$

Here, $\perp$ means no information about R_i is public, while R_i makes the entire relation public. $\sigma_\phi(R_i)$ makes public only tuples satisfying predicate ϕ , and $\pi_{B_i}(R_i)$ makes public only the boundary attributes B_i relevant to residual sensitivity. Finally, $\pi_{B_i}(\sigma_\phi(R_i))$ combines both, making public the boundary values of tuples satisfying ϕ .

Neighboring edits. We consider three elementary edits to R_i : insertion INS_i , deletion DEL_i , and value change CHG_i . Under add/remove DP, $\tau \in \{\text{INS}_i, \text{DEL}_i\}$; under change DP, $\tau = \text{CHG}_i$. We also allow combinations of these edits to capture more general neighborhood models.

Base rule. We write $\Gamma, \mathcal{A}_c(D) \vdash_I R_i : \tau \Rightarrow \ell_i$ to mean that, on database instance I , under schema constraints Γ and the information $\mathcal{A}_c(D)$ authorized for the current query, an allowed neighboring edit τ to R_i can change the query output by at most ℓ_i . When $\mathcal{A}_c(D) = \perp$, no information about R_i is public, so no refinement is possible and we recover its ordinary

²We follow the inference system of DP4SQL [11]

local-sensitivity contribution:

$$\frac{\mathcal{A}_c(D) = \perp}{\Gamma, \mathcal{A}_c(D) \vdash_I R_i : \tau \Rightarrow \text{LS}_i} \quad (\text{BASE})$$

The BASE rule therefore captures the standard DP case. The remaining rules tighten LS_i when the public view $\mathcal{A}_c(D)$ restricts the neighboring changes that are possible for R_i .

Inference rules. The box below shows the complete set of inference rules. Each rule derives the remaining sensitivity contribution ℓ_i of relation R_i under a particular public view and neighboring edit; FK additionally applies when R_i references the excluded relation R_E through a foreign key, so referential integrity eliminates the non-occurring contribution.

$\frac{\mathcal{A}_c(D) = \perp}{\Gamma, \mathcal{A}_c(D) \vdash_I R_i : \tau \Rightarrow \text{LS}_i} \quad (\text{BASE})$	
$\frac{\mathcal{A}_c(D) = \sigma_\phi(R_i) \quad \tau = \text{DEL}_i}{\Gamma, \mathcal{A}_c(D) \vdash_I R_i : \tau \Rightarrow \mu_{i,1}^{\text{free}}} \quad (\text{SEL-DEL})$	
$\frac{\mathcal{A}_c(D) = \pi_{B_i}(R_i) \quad \tau = \text{CHG}_i}{\Gamma, \mathcal{A}_c(D) \vdash_I R_i : \tau \Rightarrow 0} \quad (\text{PRJ-CHG})$	
$\frac{\mathcal{A}_c(D) = \pi_{B_i}(\sigma_\phi(R_i)) \quad \tau = \text{CHG}_i}{\Gamma, \mathcal{A}_c(D) \vdash_I R_i : \tau \Rightarrow \max\{T_{E_i} - \mu_{i,\min}^{\text{free}}, \mu_{i,1}^{\text{free}}\}} \quad (\text{PRJSEL-CHG})$	
$\frac{\tau = \text{INS}_i}{\Gamma, \mathcal{A}_c(D) \vdash_I R_i : \tau \Rightarrow \text{LS}_i} \quad (\text{INS})$	
$\frac{\mathcal{A}_c(D) = \sigma_\phi(R_i) \quad \tau = \{\text{DEL}_i, \text{CHG}_i\}}{\Gamma, \mathcal{A}_c(D) \vdash_I R_i : \tau \Rightarrow \max\{T_{E_i} - \mu_{i,\min}^{\text{free}}, \mu_{i,1}^{\text{free}}\}} \quad (\text{SEL})$	
$\frac{\mathcal{A}_c(D) = \sigma_\phi(R_i) \quad \Gamma \models \text{Unique}(B_i) \quad \tau \in \{\text{INS}_i, \text{DEL}_i, \text{CHG}_i\}}{\Gamma, \mathcal{A}_c(D) \vdash_I R_i : \tau \Rightarrow \max\{\mu_{i,1}^{\text{free}}, M_i^{\text{unocc}}\}} \quad (\text{SEL-UNIQUE})$	
$\frac{\mathcal{A}_c(D) = R_i \quad \tau \in \{\text{INS}_i, \text{DEL}_i, \text{CHG}_i\}}{\Gamma, \mathcal{A}_c(D) \vdash_I R_i : \tau \Rightarrow 0} \quad (\text{PUB})$	
$\frac{R_i \xrightarrow{\text{FK}} R_E}{\Gamma \models M_i^{\text{unocc}} = 0} \quad (\text{FK})$	

Interpreting the Rules. Recall that $T_{E_i} = \text{LS}_i$ is the largest boundary multiplicity for R_i . Among boundary values that remain private, let $\mu_{i,1}^{\text{free}}$ and $\mu_{i,\min}^{\text{free}}$ denote the largest and smallest occupied multiplicities, and let M_i^{unocc} denote the largest multiplicity at an unoccupied boundary value. The rules characterize which of these multiplicities remain reachable under contextual neighborhood. SEL-DEL excludes public tuples from deletion, leaving $\mu_{i,1}^{\text{free}}$ as the largest removable contribution. PRJ-CHG fixes all boundary values and therefore gives zero sensitivity, while PRJSEL-CHG and SEL allow changes among the remaining private values, yielding $\max\{T_{E_i} - \mu_{i,\min}^{\text{free}}, \mu_{i,1}^{\text{free}}\}$. INS retains ordinary sensitivity because a new tuple may introduce an unrestricted boundary value; under uniqueness, SEL-UNIQUE instead limits this to $\max\{\mu_{i,1}^{\text{free}}, M_i^{\text{unocc}}\}$. FK sets $M_i^{\text{unocc}} = 0$ when referential integrity prevents new boundary values. We give a detailed explanation of these rules in App. A.1. Using these rules, we

Mode	Data / Release	Pipeline	Guarantee
CI	$V_\rho^{\text{exact}} / \text{exact}$	TEXT-TO-CQL + T2SQL	CI (Thm. 5)
DP	D / noisy	DP-aware T2SQL	(ϵ, δ) -DP (Thm. 4)
CI+DP: Split	$V_\rho^{\text{exact}} / \text{exact} + \text{rest noisy}$	TEXT-TO-CQL + DP-aware T2SQL	CI + (ϵ, δ) -DP (Thms. 4, 5)
CI+DP: Context.	D / noisy	TEXT-TO-CQL + DP-aware T2SQL + CDP	$(\epsilon_{\text{syn}} + \epsilon_{\text{rel}}, \delta_{\text{syn}} + \delta_{\text{rel}})$ -CDP (Cor. 1)

Table 2: End-to-end privacy modes supported by PIMENTO

define *contextual local sensitivity* (CLS) and derive *contextual residual sensitivity* (CRS), a smooth upper bound on CLS that is no larger than standard residual sensitivity.

Theorem 3 (Contextual residual sensitivity). *For every multiway-join counting query q , database D , authorization view $\mathcal{A}_c(D)$, and smoothing parameter $\beta > 0$, $\text{CRS}_{q, \mathcal{A}_c(D)}(D)$ is a β -smooth upper bound on $\text{CLS}_{q, \mathcal{A}_c(D)}(D)$ and satisfies*

$$\text{CRS}_{q, \mathcal{A}_c(D)}(D) \leq \text{RS}_q(D).$$

Consequently, calibrating the release mechanism to $\text{CRS}_{q, \mathcal{A}_c(D)}(D)$ yields (ϵ, δ) -contextual DP under $\mathcal{A}_c(D)$. The construction and proof are given in App. A.2.

Corollary 1 (End-to-end privacy). *Since (ϵ, δ) -DP implies (ϵ, δ) -contextual DP under any authorization view $\mathcal{A}_c(D)$, if the synthetic release is $(\epsilon_{\text{syn}}, \delta_{\text{syn}})$ -DP and the final query release is $(\epsilon_{\text{rel}}, \delta_{\text{rel}})$ -contextual DP under $\mathcal{A}_c(D)$, then PIMENTO is $(\epsilon_{\text{syn}} + \epsilon_{\text{rel}}, \delta_{\text{syn}} + \delta_{\text{rel}})$ -contextual DP under $\mathcal{A}_c(D)$. The proof is given in App. A.2.*

4.4 End-to-End Privacy

We have developed three components of PIMENTO: DP-aware Text-to-SQL for query generation, CI-aware Text-to-SQL for policy compilation, and contextual DP (CDP). Together, they provide three privacy-preserving release modes, exposed as a choice to the application. All modes receive the same natural-language inputs: the text corpus, privacy policy $\mathcal{P}$, and query.

CI Only. When the application needs contextual access control but permits exact disclosure, PIMENTO instantiates the CI views for the query context and computes the authorized view V_ρ^{exact} . The query is evaluated against V_ρ^{exact} , so only authorized data is released exactly, satisfying CI by Thm. 5.

DP Only. When the full population should be protected uniformly, PIMENTO uses DP-aware Text-to-SQL to select a correct, low-sensitivity formulation q , and releases a noisy answer satisfying standard DP by Thm. 4.

CI+DP. When a query spans both CI-authorized and private information, PIMENTO first derives the authorized view V_ρ^{exact} and supports two release semantics. (i) *Split release* treats the

Table 3: Text-to-SQL performance on DP-D1. T1/T10 denote top-1/top-10 candidates; Joint is Is-Min $\wedge$ Acc.; and Δ is the Joint improvement over zero-shot T1. **Bold** marks the best result and underline the best baseline.

Method	Acc.	Is-Min	Joint	Δ
<i>XiYanSQL-32B</i>				
Zero-shot	77.7/70.3	50.5/55.5	49.1/52.8	0.0/+3.7
Sens. prompt	64.0/68.2	38.7/50.5	37.2/47.9	-11.9/-1.2
SFT	78.9/81.5	50.1/62.7	47.5/59.9	-1.6/+10.8
DPO	67.3/65.1	37.9/47.0	36.9/44.1	-12.2/-5.0
DPO on SFT	73.2/73.9	45.7/49.2	45.2/48.5	-3.9/-0.6
AlphaSQL	80.4	39.8	35.6	-13.5
PIMENTO	81.8	76.6	75.3	+26.2
<i>GPT-5.6-Luna</i>				
Zero-shot	79.7/79.7	32.1/39.8	28.7/35.6	0.0/+6.9
Sens. prompt	76.0/76.6	30.1/39.5	30.1/38.5	+1.4/+9.8
AlphaSQL	84.2	47.9	43.9	+15.2
PIMENTO	83.0	77.2	74.6	+45.9
<i>Phi-4</i>				
Zero-shot	71.3/72.9	46.4/ <u>53.3</u>	42.1/ <u>47.9</u>	0.0/+5.8
Sens. prompt	63.4/65.9	30.6/43.3	28.6/38.6	-13.5/-3.5
AlphaSQL	<u>77.4</u>	41.2	34.9	-7.2
PIMENTO	79.6	74.1	71.2	+29.1

Table 4: TEXT-TO-CQL performance across datasets.

Dataset	Dimension	Precision	Recall	F1
CONTEXT-D1	Role	62.1%	61.1%	61.5%
	Data	83.3%	97.3%	86.5%
	Conditions	95.6%	76.6%	76.3%
	All	68.2%	70.3%	68.0%
CONTEXT-D2	Role	96.1%	68.0%	71.7%
	Data	77.4%	71.8%	70.0%
	Conditions	83.7%	67.2%	68.2%
	All	91.1%	68.9%	71.2%
CONTEXT-D3	Role	100.0%	81.7%	88.6%
	Data	95.1%	100.0%	97.3%
	Conditions	97.9%	82.1%	85.9%
	All	98.6%	85.4%	89.7%

portions separately: CI-authorized information is released exactly, while the remainder is released under standard DP. This applies when the application can meaningfully return the two parts separately. (ii) *Contextual DP* instead releases a single statistic over the full query. It holds the authorized information $\mathcal{A}_c(D)$ fixed in the neighboring relation and calibrates noise using contextual residual sensitivity, potentially requiring less noise than standard DP. By Cor. 1, if synthetic-data generation is $(\epsilon_{\text{syn}}, \delta_{\text{syn}})$ -DP and the final release is $(\epsilon_{\text{rel}}, \delta_{\text{rel}})$ -contextual DP, the complete pipeline is $(\epsilon_{\text{syn}} + \epsilon_{\text{rel}}, \delta_{\text{syn}} + \delta_{\text{rel}})$ -contextual DP. Tab. 2 summarizes the guarantees.

When to use each mode. The choice depends on the intended release. In our hospital example, *CI only* mode lets a resident receive authorized diagnoses for their care-team patients exactly. *DP only* mode instead supports aggregates e.g., “How many patients have HIV?”, protecting every patient with noise. *CI+DP with split release* applies when the portions can be separated: a doctor may receive exact statistics for their own patients and a DP-protected aggregate for the rest. *Contextual DP* applies when a single aggregate is needed: a researcher authorized to access their study patients can receive one hospital-wide noisy count, with noise calibrated only to records they are not authorized to see.

5 Evaluation

Our evaluation is organized around the following questions:

- **RQ1.** Can PIMENTO find correct SQL with lower sensitivity than existing methods?
- **RQ2.** Can PIMENTO perform TEXT-TO-CQL, faithfully translating CI policies into accurate SQL views?
- **RQ3.** Does contextual DP reduce noise on contextually authorized information compared to standard DP?
- **RQ4.** When composed end to end, does PIMENTO adhere more faithfully to the policy than baselines?

Benchmarks. No existing benchmark jointly provides natural language (i) text, (ii) queries, and (iii) policies together with ground truth databases, SQL queries annotated for sensitivity, query contexts, and policy grounded SQL views. We therefore construct two benchmarks by combining existing datasets with new annotations. DP-D1 contains natural language questions paired with multiple correct SQL formulations, their Residual Sensitivity values, and the minimum sensitivity formulation q^* , with 3,673 training and 886 test questions. Our CI benchmark contains 450 query context instances across healthcare (CONTEXT-D1), consumer (CONTEXT-D2), and children’s privacy (CONTEXT-D3), with 150 instances each. Each instance includes the natural language data, privacy policy, query, runtime context, and ground truth authorized output. We use DP-D1 for RQ1 and CONTEXT-D1–CONTEXT-D3 for RQ2–RQ4. See App. A.4 for details.

5.1 RQ1: DP-Aware Text2SQL

Setup. We use $(\epsilon_{\text{syn}}, \delta_{\text{syn}}) = (1.6, 10^{-6})$ for synthetic data and $(\epsilon, \delta) = (1.0, 10^{-6})$ per query, with 10 rollouts and expansion size 5. We evaluate XIYANSQL-QWENCODER-32B, PHI-4, and GPT-5.6-LUNA against **six** baselines: *Zero-shot*; *Sensitivity Prompt*, which prompts for low sensitivity; *Fine-tuning* on XiYanSQL-32B, including SFT [63], DPO [64], and DPO on SFT; *Top-k Selection* ($k = 10$), which selects the lowest-sensitivity candidate; and *AlphaSQL* [44], which uses MCTS without our sensitivity-guided search. We report execution accuracy (*Acc.*), minimum-sensitivity rate (*Is-Min*), and their intersection (*Joint*) in Tab. 3. DP noise reduction, inference costs, and ablations are reported in App. A.5.

Results. Table 3 shows that PIMENTO substantially improves the joint objective across all three model families, achieving 75.3%, 74.6%, and 71.2% Joint accuracy with XiYanSQL-32B, GPT-5.6-Luna, and Phi-4, respectively. This improves over zero-shot by 26.2, 45.9, and 29.1 points, and over the strongest baseline by 15.4, 30.7, and 23.3 points. In contrast, AlphaSQL achieves high accuracy but achieves only 39.8–47.9% Is-Min. Thus, search alone is insufficient; explicitly steering it toward low-sensitivity formulations is necessary to jointly preserve correctness and DP utility.

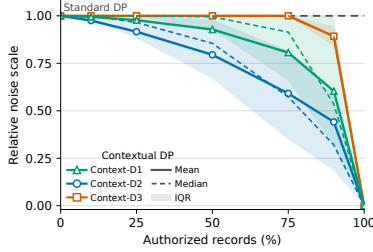


Figure 4: Relative noise required by contextual DP as we authorize an increasing fraction of subject records.

5.2 RQ2: CI-Aware Text2SQL

Setup. We evaluate whether PIMENTO can compile natural-language CI norms into SQL views, i.e., TEXT-TO-CQL, by comparing the compiled views against manually annotated ground truth. We report precision, recall, and F1 scores.

Results. Tab. 4 shows high precision across all mapping types (62.1–100%), with lower recall indicating that PIMENTO more often misses valid mappings than introduces incorrect ones. Performance is strongest on CONTEXT-D3, reaching F1 scores of 88.6%, 97.3%, and 85.9% for role, data, and condition mappings, respectively.

5.3 RQ3: Contextual DP

Setup. In Sec. 4.3, we characterize the conditions under which contextual DP reduces noise. Since these patterns do not naturally arise under our benchmark policies, we validate the derived cases by progressively authorizing records in each dataset’s primary subject relation and comparing the average required noise across all queries under contextual and standard DP with the same privacy budget, $\epsilon = 1$.

Results. Fig. 4 validates our characterization: contextual DP never requires more noise than standard DP. At 90% authorization, mean noise falls to $0.44\times$ for CONTEXT-D2, $0.60\times$ for CONTEXT-D1, and $0.89\times$ for CONTEXT-D3, corresponding to reductions of 56%, 40%, and 11%, respectively.

5.4 RQ4: End-to-End Evaluation

Setup. We evaluate PIMENTO end to end, from natural-language data and policy to the final answer released to the user. We compare the resulting *information leakage* under four approaches using GPT-5.6-Luna: *Text-only*, which answers directly from the text without the policy; *Text+Policy*, which gives the LLM the policy and context and asks it to enforce them; *AirGapAgent* [5], which first uses a trusted LLM to filter the text to information it considers authorized; and PIMENTO. We report *leakage rate*, the fraction of responses that reveal information prohibited by the ground-truth policy for that query.

Results. Tab. 5 shows a clear reduction in leakage as enforcement becomes more explicit. Giving the policy directly to the

System	CONTEXT-D1	CONTEXT-D2	CONTEXT-D3
Text-only	88.7%	85.2%	100.0%
Text+Policy	18.0%	52.3%	44.3%
AirGapAgent	6.7%	38.9%	7.4%
PIMENTO	0.0%	2.1%	1.3%
PIMENTO [†]	0.0%	0.0%	0.0%

Table 5: End-to-end prohibited information leakage using GPT-5.6-Luna. PIMENTO[†] uses the ground-truth policy views. Leakage in PIMENTO can additionally be protected under DP rather than released exactly.

LLM (Text+Policy) still leaks prohibited information in 18.0–52.3% of queries, while AirGapAgent reduces this to 6.7–38.9%. In contrast, PIMENTO reduces leakage to 0.0%, 2.1%, and 1.3% on CONTEXT-D1, CONTEXT-D2, and CONTEXT-D3, respectively. The leakage in PIMENTO comes entirely from policy-view compilation errors: with ground-truth views, deterministic enforcement achieves 0% leakage on all datasets. Moreover, the leakage in PIMENTO can still be protected under DP rather than released exactly.

6 Related Work

CI for text. Prior work extracts or annotates CI norms from natural-language policies [12, 69–71], uses CI as LLM context for leakage detection [5, 23, 26, 34, 43, 45, 46, 58, 79, 85]. PIMENTO instead compiles CI policies into SQL views.

DP for text. DP for text [33, 40] spans local sanitization of text [4, 13, 24, 25, 51, 53, 86, 93], private language-model training [2, 47, 52, 88], and private synthetic-text generation [50, 91], often requiring substantial noise that degrades utility [90]. PIMENTO instead grounds text into a relational representation and privatizes query answers. Prior SQL-DP work optimizes execution plans, query rewrites, sensitivity bounds, or DP mechanisms [18–20, 37, 38, 42]. PIMENTO instead applies DP to SQL queries, building on prior work on private query answering [19, 38, 42], but uniquely optimizes privacy by searching over query formulations from natural language.

Policy-aware DP. Customized DP adapts protection using explicit secrets or constraints [28, 30, 39], sensitive records or text spans [68], known relational privacy policies [11]. These approaches take protection semantics as given; PIMENTO instead derives querier- and context-specific authorization from natural-language CI policies and uses it to define the DP neighborhood. Prior work has connected CI and DP conceptually [8]; PIMENTO makes this connection operational.

7 Conclusion

We presented PIMENTO, a framework for formally enforcing CI and DP over unstructured text through a shared relational representation. PIMENTO supports DP aware query generation, executable CI policies, and their composition through *contextual differential privacy*.

Ethical Considerations

This paper studies privacy-preserving analytics over sensitive unstructured text using contextual integrity (CI), differential privacy (DP), or their composition. Because the target setting includes domains such as healthcare and consumer data, failures may expose sensitive information. We therefore consider the ethical implications of both the system and its deployment. **Stakeholders.** Stakeholders include individuals represented in the underlying text, data custodians, analysts issuing queries, administrators defining privacy policies, and the research community.

Principles. We consider the Menlo Report principles of Beneficence, Respect for Persons, Justice, and Respect for Law and Public Interest. Our goal is to enable useful analysis while limiting inappropriate information flows and inference about individuals.

Potential harms. The primary risk is unintended disclosure of sensitive information. While PIMENTO enforces CI and DP through explicit mechanisms rather than LLM judgments, errors in grounding natural-language data, queries, or policies may still affect correctness or privacy.

The system also has potential for dual use. Privacy-preserving access to sensitive organizational text could be deployed under overly permissive policies or where affected individuals have limited control over how their data is analyzed. Formal enforcement guarantees adherence to the supplied policy, not that the policy itself is ethically appropriate.

Mitigations. We mitigate these risks by making privacy enforcement explicit and auditable. CI policies are compiled into database views that can be inspected and validated before deployment, while runtime authorization is enforced deterministically. DP releases provide formally bounded disclosure, and contextual DP applies only after determining the information authorized for the requesting context.

Our experiments do not recruit or interact with human participants. We evaluate on existing Text-to-SQL benchmarks and public privacy policies, including HIPAA, CCPA, and COPPA, with additional annotations for evaluating privacy enforcement.

Decision. We believe it is ethical to proceed and publish because the work aims to reduce privacy risks in natural-language analytics while making its assumptions and failure modes explicit. Publication enables scrutiny of these guarantees and provides a more auditable alternative to approaches that delegate privacy decisions directly to LLMs.

References

- [1] Sahar Abdelnabi, Amr Gomaa, Eugene Bagdasarian, Per Ola Kristensson, and Reza Shokri. Firewalls to secure dynamic llm agentic networks, 2025. URL <https://arxiv.org/abs/2502.1822>.
- [2] Rohan Anil, Badih Ghazi, Vineet Gupta, Ravi Kumar, and Pasin Manurangsi. Large-scale differentially private BERT. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, editors, *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 6481–6491, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. URL: <https://aclanthology.org/2022.findings-emnlp.484/>, doi:10.18653/v1/2022.findings-emnlp.484.
- [3] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avani Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. Language models enable simple systems for generating structured views of heterogeneous data lakes, 2025. URL: <https://arxiv.org/abs/2304.09433>, arXiv:2304.09433.
- [4] Ahmed Musa Awon, Yun Lu, Shera Potka, and Alex Thomo. CluSanT: Differentially private and semantically coherent text sanitization. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3676–3693, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. URL: <https://aclanthology.org/2025.naacl-long.187/>, doi:10.18653/v1/2025.naacl-long.187.
- [5] Eugene Bagdasarian, Ren Yi, Sahra Ghalebikesabi, Peter Kairouz, Marco Gruteser, Sewoong Oh, Borja Balle, and Daniel Ramage. Airgapagent: Protecting privacy-conscious conversational agents. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 3868–3882, 2024.
- [6] Adam Barth, Anupam Datta, John C Mitchell, and Helen Nissenbaum. Privacy and contextual integrity: Framework and applications. In *2006 IEEE Symposium on Security and Privacy*, pages 184–198. IEEE, 2006. doi:10.1109/SP.2006.32.
- [7] Johes Bater, Xi He, William Ehrich, Ashwin Machanavajjhala, and Jennie Rogers. Shrinkwrap: efficient sql query processing in differentially private data federations. *Proceedings of the VLDB Endowment*, 12(3), 2018.
- [8] Sebastian Benthall and Rachel Cummings. Integrating differential privacy and contextual integrity. In *Proceedings of the 2024 Symposium on Computer Science and Law*, pages 9–15, 2024.
- [9] Adithya Bhaskar, Tushar Tomar, Ashutosh Sathe, and Sunita Sarawagi. Benchmarking and improving text-to-sql generation under ambiguity. In *Proceedings of*

the 2023 Conference on Empirical Methods in Natural Language Processing, pages 7053–7074, 2023.

- [10] Kuntai Cai, Xiaokui Xiao, and Yin Yang. PrivPetal: Relational data synthesis via permutation relations. *Proceedings of the ACM on Management of Data (SIGMOD)*, 3(3), 2025. doi:10.1145/3725341.
- [11] Andrew Cascio, KinChin Tong, Daniel Kifer, Zeyu Ding, and Danfeng Zhang. Dp4sql: Differentially private sql with flexible privacy policies. *arXiv preprint arXiv:2606.07883*, 2026.
- [12] Jake Chanenson, Madison Pickering, and Noah Apthorpe. Automating governing knowledge commons and contextual integrity (gkc-ci) privacy policy annotations with large language models. *arXiv preprint arXiv:2311.02192*, 2023.
- [13] Amrita Roy Chowdhury, David Glukhov, Divyam Anshuman, Prasad Chalasani, Nicolas Papernot, Somesh Jha, and Mihir Bellare. Preempt: Sanitizing sensitive prompts for llms. In *Network and Distributed System Security Symposium (NDSS)*, 2026. doi:10.14722/n.dss.2026.231277.
- [14] Congruity360. Ccpa compliance guide. URL: <https://www.congruity360.com/blog/ccpa-compliance-guide/>.
- [15] Zheyang Deng, Chunkit Chan, Weiqi Wang, Yuxi Sun, Wei Fan, Tianshi Zheng, Yauwai Yim, and Yangqiu Song. Text-tuple-table: Towards information integration in text-to-table generation via global tuple extraction. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9300–9322, Miami, Florida, USA, November 2024. Association for Computational Linguistics. URL: <https://aclanthology.org/2024.emnlp-main.523/>, doi:10.18653/v1/2024.emnlp-main.523.
- [16] Zhongjun Ding, Yin Lin, Tianjing Zeng, Rong Zhu, Bolin Ding, and Jingren Zhou. Ambisql: Interactive ambiguity detection and resolution for text-to-sql. In *Companion of the International Conference on Management of Data*, pages 26–29, 2026.
- [17] Mingwen Dong, Nischal Ashok Kumar, Yiqun Hu, Anuj Chauhan, Chung-Wei Hang, Shuaichen Chang, Lin Pan, Wuwei Lan, Henghui Zhu, Jiarong Jiang, et al. Practiq: A practical conversational text-to-sql dataset with ambiguous and unanswerable queries. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 255–273, 2025.
- [18] Wei Dong, Juanru Fang, Ke Yi, Yuchao Tao, and Ashwin Machanavajjhala. R2t: Instance-optimal truncation for differentially private query evaluation with foreign keys. In *Proceedings of the 2022 International Conference on Management of Data*, pages 759–772, 2022.
- [19] Wei Dong and Ke Yi. Residual sensitivity for differentially private multi-way joins. In *Proceedings of the 2021 International Conference on Management of Data*, pages 432–444. ACM, 2021. doi:10.1145/3448016.3452813.
- [20] Wei Dong and Ke Yi. A nearly instance-optimal differentially private mechanism for conjunctive queries. In *Proceedings of the 41st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, pages 213–225, 2022.
- [21] Cynthia Dwork. Differential privacy. In *International colloquium on automata, languages, and programming*, pages 1–12. Springer, 2006.
- [22] Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3–4):211–407, 2014. doi:10.1561/04000000042.
- [23] Wei Fan, Haoran Li, Zheyang Deng, Weiqi Wang, and Yangqiu Song. Goldcoin: Grounding large language models in privacy laws via contextual integrity theory. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 3321–3343, 2024.
- [24] Oluwaseyi Feyisetan, Borja Balle, Thomas Drake, and Tom Diethe. Privacy-and utility-preserving textual analysis via calibrated multivariate perturbations. In *Proceedings of the 13th international conference on web search and data mining*, pages 178–186, 2020.
- [25] Oluwaseyi Feyisetan and Shiva Kasiviswanathan. Private release of text embedding vectors. In Yada Pruksachatkun, Anil Ramakrishna, Kai-Wei Chang, Satyapriya Krishna, Jwala Dhamala, Tanaya Guha, and Xiang Ren, editors, *Proceedings of the First Workshop on Trustworthy Natural Language Processing*, pages 15–27, Online, June 2021. Association for Computational Linguistics. URL: <https://aclanthology.org/2021.trustnlp-1.3/>, doi:10.18653/v1/2021.trustnlp-1.3.
- [26] Sahra Ghalebikesabi, Eugene Bagdasarian, Ren Yi, Itay Yona, Ilia Shumailov, Aneesh Pappu, Chongyang Shi, Laura Weidinger, Robert Stanforth, Leonard Berrada, et al. Privacy awareness for information-sharing assistants: A case-study on form-filling with contextual integrity. *Transactions on Machine Learning Research*, 2025.

- [27] Google Cloud. Document ai overview. Google Cloud Documentation, 2026. Accessed: 2026-08-24. URL: <https://docs.cloud.google.com/document-ai/docs/overview>.
- [28] Samuel Haney, Ashwin Machanavajjhala, and Bolin Ding. Design of policy-aware differentially private algorithms. *arXiv preprint arXiv:1404.3722*, 2014.
- [29] Tam Harbert. <https://mitsloan.mit.edu/ideas-made-to-matter/tapping-power-unstructured-data>.
- [30] Xi He, Ashwin Machanavajjhala, and Bolin Ding. Blowfish privacy: Tuning privacy-utility trade-offs using policies. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*, pages 1447–1458, 2014.
- [31] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. Next-generation database interfaces: A survey of llm-based text-to-sql. *IEEE Transactions on Knowledge and Data Engineering*, 2025.
- [32] Lijie Hu, Ivan Habernal, Lei Shen, and Di Wang. Differentially private natural language models: Recent advances and future directions. In Yvette Graham and Matthew Purver, editors, *Findings of the Association for Computational Linguistics: EACL 2024*, pages 478–499, St. Julian’s, Malta, March 2024. Association for Computational Linguistics. URL: <https://aclanthology.org/2024.findings-eacl.33/>, doi:10.18653/v1/2024.findings-eacl.33.
- [33] Lijie Hu, Ivan Habernal, Lei Shen, and Di Wang. Differentially private natural language models: Recent advances and future directions. In Yvette Graham and Matthew Purver, editors, *Findings of the Association for Computational Linguistics: EACL 2024*, pages 478–499, St. Julian’s, Malta, March 2024. Association for Computational Linguistics. URL: <https://aclanthology.org/2024.findings-eacl.33/>, doi:10.18653/v1/2024.findings-eacl.33.
- [34] Xinyue Huang, Xiaochun Cao, and Wenyuan Yang. Need to know: Contextual-integrity-grounded query rewriting for privacy-conscious llm delegation, 2026. URL: <https://arxiv.org/abs/2606.04067>, arXiv:2606.04067.
- [35] Timour Igamberdiev and Ivan Habernal. Dp-bart for privatized text rewriting under local differential privacy. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13914–13934, 2023.
- [36] Yizhu Jiao, Sha Li, Sizhe Zhou, Heng Ji, and Jiawei Han. Text2DB: Integration-aware information extraction with large language model agents. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 185–205, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL: <https://aclanthology.org/2024.findings-acl.12/>, doi:10.18653/v1/2024.findings-acl.12.
- [37] Noah Johnson, Joseph P Near, Joseph M Hellerstein, and Dawn Song. Chorus: a programming framework for building scalable differential privacy mechanisms. In *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 535–551. IEEE, 2020.
- [38] Noah Johnson, Joseph P Near, and Dawn Song. Towards practical differential privacy for sql queries. *Proceedings of the VLDB Endowment*, 11(5):526–539, 2018.
- [39] Daniel Kifer and Ashwin Machanavajjhala. Pufferfish: A framework for mathematical privacy definitions. *ACM Transactions on Database Systems (TODS)*, 39(1):1–36, 2014.
- [40] Oleksandra Klymenko, Stephen Meisenbacher, and Florian Matthes. Differential privacy in natural language processing: The story so far. In Oluwaseyi Feyisetan, Sepideh Ghanavati, Patricia Thaine, Ivan Habernal, and Fatemehsadat Mireshghallah, editors, *Proceedings of the Fourth Workshop on Privacy in Natural Language Processing*, pages 1–11, Seattle, United States, July 2022. Association for Computational Linguistics. URL: <http://aclanthology.org/2022.privatenlp-1.1/>, doi:10.18653/v1/2022.privatenlp-1.1.
- [41] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *Machine Learning: ECML 2006*, volume 4212 of *Lecture Notes in Computer Science*, pages 282–293. Springer, 2006. doi:10.1007/11871842_29.
- [42] Ios Kotsogiannis, Yuchao Tao, Xi He, Maryam Fanaeepour, Ashwin Machanavajjhala, Michael Hay, and Jerome Miklau. Privatesql: a differentially private sql query engine. *Proceedings of the VLDB Endowment*, 12(11):1371–1384, 2019.
- [43] Guangchen Lan, Huseyin A Inan, Sahar Abdelnabi, Janardhan Kulkarni, Lukas Wutschitz, Reza Shokri, Christopher G Brinton, and Robert Sim. Contextual integrity in llms via reasoning and reinforcement learning. *arXiv preprint arXiv:2506.04245*, 2025.
- [44] Boyan Li, Jiayi Zhang, Ju Fan, Yanwei Xu, Chong Chen, Nan Tang, and Yuyu Luo. Alpha-SQL: Zero-shot text-to-sql using monte carlo tree search. *arXiv preprint arXiv:2502.17248*, 2025.

- [45] Haoran Li, Wei Fan, Yulin Chen, Cheng Jiayang, Tianshu Chu, Xuebing Zhou, Peizhao Hu, and Yangqiu Song. Privacy checklist: Privacy violation detection grounding on contextual integrity theory. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1748–1766, 2025.
- [46] Haoran Li, Wenbin Hu, Huihao Jing, Yulin Chen, Qi Hu, Sirui Han, Tianshu Chu, Peizhao Hu, and Yangqiu Song. PrivaCI-bench: Evaluating privacy with contextual integrity and legal compliance. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10544–10559, Vienna, Austria, July 2025. Association for Computational Linguistics. URL: <https://aclanthology.org/2025.acl-long.518/>, doi:10.18653/v1/2025.acl-long.518.
- [47] Xuechen Li, Florian Tramer, Percy Liang, and Tatsunori Hashimoto. Large Language Models Can Be Strong Differentially Private Learners. In *International Conference on Learning Representations*, 2022. URL: <https://mlanthology.org/iclr/2022/li2022iclr-large/>.
- [48] Zhuowen Liang, Zhengxuan Zhang, Jiayang Wang, Jiazhuo Chen, and Nan Tang. Beyond tables: Doc2db-bench for relationally faithful document-to-database construction. *arXiv preprint arXiv:2608.08459*, 2026.
- [49] Teng Lin, Yuyu Luo, and Nan Tang. Structure then query: Enabling precise analytical queries over unstructured documents. *arXiv preprint arXiv:2608.13384*, 2026.
- [50] Justus Mattern, Zhijing Jin, Benjamin Weggenmann, Bernhard Schölkopf, and Mrinmaya Sachan. Differentially private language models for secure data sharing. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 4860–4873, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. URL: <https://aclanthology.org/2022.emnlp-main.323/>, doi:10.18653/v1/2022.emnlp-main.323.
- [51] Justus Mattern, Benjamin Weggenmann, and Florian Kerschbaum. The limits of word level differential privacy. In Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz, editors, *Findings of the Association for Computational Linguistics: NAACL 2022*, pages 867–881, Seattle, United States, July 2022. Association for Computational Linguistics. URL: <https://aclanthology.org/2022.findings-naacl.65/>, doi:10.18653/v1/2022.findings-naacl.65.
- [52] Brendan McMahan, Daniel Ramage, Kunal Talwar, and Li Zhang. Learning differentially private recurrent language models. In *International Conference on Learning Representations (ICLR)*, 2018. URL: <https://openreview.net/pdf?id=BJ0hF1Z0b>.
- [53] Casey Meehan, Khalil Mrini, and Kamalika Chaudhuri. Sentence-level privacy for document embeddings. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3367–3380, Dublin, Ireland, May 2022. Association for Computational Linguistics. URL: <https://aclanthology.org/2022.acl-long.238/>, doi:10.18653/v1/2022.acl-long.238.
- [54] Stephen Meisenbacher, Maulik Chevli, and Florian Matthes. Leveraging semantic triples for private document generation with local differential privacy guarantees, 2025. URL: <https://arxiv.org/abs/2508.20736>, arXiv:2508.20736.
- [55] Stephen Meisenbacher, Nihildev Nandakumar, Alexandra Klymenko, and Florian Matthes. A comparative analysis of word-level metric differential privacy: Benchmarking the privacy-utility trade-off. In Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue, editors, *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 174–185, Torino, Italia, May 2024. ELRA and ICCL. URL: <https://aclanthology.org/2024.lrec-main.16/>.
- [56] Microsoft. Choose an azure ai targeted language processing technology. Microsoft Learn, 2026. Accessed: 2026-08-24. URL: <https://learn.microsoft.com/en-us/azure/architecture/data-guide/ai-services/targeted-language-processing>.
- [57] Microsoft. What is azure document intelligence in foundry tools? Microsoft Learn, 2026. Accessed: 2026-08-24. URL: <https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/overview?view=doc-intel-4.0.0>.
- [58] Niloofer Miresghallah, Hyunwoo Kim, Xuhui Zhou, Yulia Tsvetkov, Maarten Sap, Reza Shokri, and Yejin Choi. Can llms keep a secret? testing privacy implications of language models via contextual integrity theory. In *International Conference on Learning Representations*, 2024.

- [59] Ali Mohammadjafari, Anthony S Maida, and Raju Gottumukkala. From natural language to sql: Review of llm-based text-to-sql systems. *arXiv preprint arXiv:2410.01066*, 2024.
- [60] Ivoline C Ngong, Swanand Ravindra Kadhe, Hao Wang, Keerthiram Murugesan, Justin D Weisz, Amit Dhurandhar, and Karthikeyan Natesan Ramamurthy. Protecting users from themselves: Safeguarding contextual privacy in interactions with conversational agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 26196–26220, 2025.
- [61] Kobbi Nissim, Sofya Raskhodnikova, and Adam Smith. Smooth sensitivity and sampling in private data analysis. In *Proceedings of the thirty-ninth annual ACM symposium on Theory of computing*, pages 75–84, 2007.
- [62] Simone Papicchio, Luca Cagliero, and Paolo Papotti. Evaluating ambiguous questions in text2sql. In *ELLIS workshop on Representation Learning and Generative Models for Structured Data*.
- [63] Aldo Pareja, Nikhil Shivakumar Nayak, Hao Wang, Krishnateja Killamsetty, Shivchander Sudalairaj, Wenlong Zhao, Seungwook Han, Abhishek Bhandwalder, Guangxuan Xu, Kai Xu, Ligong Han, Luke Inglis, and Akash Srivastava. Unveiling the secret recipe: A guide for supervised fine-tuning small llms, 2024. URL: <https://arxiv.org/abs/2412.13337>, arXiv: 2412.13337.
- [64] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- [65] Reuters. Little breathing room: California privacy agency modifies proposed regulations, December 2022. URL: <https://www.reuters.com/legal/legalindustry/little-breathing-room-california-privacy-agency-modifies-proposed-regulations-2022-12-08/>.
- [66] Mushtari Sadia, Zhenning Yang, Yunming Xiao, Ang Chen, and Amrita Roy Chowdhury. SQUiD: Synthesizing relational databases from unstructured text. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 31987–32012, Suzhou, China, November 2025. Association for Computational Linguistics. URL: <https://aclanthology.org/2025.emnlp-main.1629/>, doi:10.18653/v1/2025.emnlp-main.1629.
- [67] Liang Shi, Zhengju Tang, Nan Zhang, Xiaotong Zhang, and Zhi Yang. A survey on employing large language models for text-to-sql tasks. *ACM Computing Surveys*, 58(2):1–37, 2025.
- [68] Weiyan Shi, Aiqi Cui, Evan Li, Ruoxi Jia, and Zhou Yu. Selective differential privacy for language modeling. In Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz, editors, *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2848–2859, Seattle, United States, July 2022. Association for Computational Linguistics. URL: <https://aclanthology.org/2022.naacl-main.205/>, doi:10.18653/v1/2022.naacl-main.205.
- [69] Yan Shvartzshanider, Ananth Balashankar, Thomas Wies, and Lakshminarayanan Subramanian. RECIPE: Applying open domain question answering to privacy policies. In Eunsol Choi, Minjoon Seo, Danqi Chen, Robin Jia, and Jonathan Berant, editors, *Proceedings of the Workshop on Machine Reading for Question Answering*, pages 71–77, Melbourne, Australia, July 2018. Association for Computational Linguistics. URL: <https://aclanthology.org/W18-2608/>, doi:10.18653/v1/W18-2608.
- [70] Yan Shvartzshanider, Ananth Balashankar, Thomas Wies, and Lakshminarayanan Subramanian. Beyond the text: Analysis of privacy statements through syntactic and semantic role labeling. In Daniel Preotiu-Pietro, Catalina Goanta, Ilias Chalkidis, Leslie Barrett, Gerasimos Spanakis, and Nikolaos Aletras, editors, *Proceedings of the Natural Legal Language Processing Workshop 2023*, pages 85–98, Singapore, December 2023. Association for Computational Linguistics. URL: <https://aclanthology.org/2023.nllp-1.10/>.
- [71] Yan Shvartzshnaider, Noah Apthorpe, Nick Fearnster, and Helen Nissenbaum. Going against the (appropriate) flow: A contextual integrity approach to privacy policy analysis. In *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing*, volume 7, pages 162–170, 2019.
- [72] Yan Shvartzshnaider and Vasisht Duddu. Position: Contextual integrity is inadequately applied to language models. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 82200–82210. PMLR, 13–19 Jul 2025. URL: <https://proceedings.mlr.press/v267/shvartzshnaider25a.html>.

- [73] Li Siyan, Vethavikashini Chithrara Raghuram, Omar Khattab, Julia Hirschberg, and Zhou Yu. Papiilon: Privacy preservation from internet-based and local language model ensembles. *arXiv preprint arXiv:2410.17127*, 2024.
- [74] Snowflake. Unstructured data analytics and ai solutions. Snowflake, 2026. Accessed: 2026-08-24. URL: <https://www.snowflake.com/en/product/use-cases/unstructured-data-analytics/>.
- [75] Talonic. Talonic: Ingest once. query forever. Talonic, 2026. Accessed: 2026-08-24. URL: <https://talonic.com/>.
- [76] Lillian Tsai and Eugene Bagdasarian. Contextual agent security: A policy for every purpose. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems*, pages 8–17, 2025.
- [77] Unstract. Unstract: Agentic document processing and extraction platform. AWS Marketplace, 2026. Accessed: 2026-08-24. URL: <https://aws.amazon.com/marketplace/pp/prodview-jlud5hmy2npui>.
- [78] Unstructured. Introducing: Extract. Unstructured, 2026. Accessed: 2026-08-24. URL: <https://unstructured.io/blog/introducing-extract>.
- [79] Shouju Wang, Fenglin Yu, Xirui Liu, Xiaoting Qin, Jue Zhang, Qingwei Lin, Dongmei Zhang, and Saravan Rajmohan. Privacy in action: Towards realistic privacy mitigation and evaluation for LLM-powered agents. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 17055–17074, Suzhou, China, November 2025. Association for Computational Linguistics. URL: <https://aclanthology.org/2025.findings-emnlp.925/>, doi:10.18653/v1/2025.findings-emnlp.925.
- [80] Primal Wijesekera, Joel Reardon, Irwin Reyes, Lynn Tsai, Jung-Wei Chen, Nathan Good, David Wagner, Konstantin Beznosov, and Serge Egelman. Contextualizing privacy decisions for better prediction (and protection). In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, CHI ’18, page 1–13, New York, NY, USA, 2018. Association for Computing Machinery. doi:10.1145/3173574.3173842.
- [81] Wikipedia contributors. General data protection regulation. Wikipedia, The Free Encyclopedia, 2026. Accessed: 2026-08-24. URL: https://en.wikipedia.org/wiki/General_Data_Protection_Regulation.
- [82] Wikipedia contributors. Health insurance portability and accountability act. Wikipedia, The Free Encyclopedia, 2026. Accessed: 2026-08-24. URL: https://en.wikipedia.org/wiki/Health_Insurance_Portability_and_Accountability_Act.
- [83] Wikipedia contributors. Shapley value. https://en.wikipedia.org/wiki/Shapley_value, 2026. Wikipedia, The Free Encyclopedia.
- [84] Royce J Wilson, Celia Yuxin Zhang, William Lam, Damien Desfontaines, Daniel Simmons-Marengo, and Bryant Gipson. Differentially private sql with bounded user contribution. *arXiv preprint arXiv:1909.01917*, 2019.
- [85] Yijia Xiao, Yiqiao Jin, Yushi Bai, Yue Wu, Xianjun Yang, Xiao Luo, Wenchao Yu, Xujiang Zhao, Yanchi Liu, Quanquan Gu, Haifeng Chen, Wei Wang, and Wei Cheng. Large language models can be contextual privacy protection learners. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 14179–14201, Miami, Florida, USA, November 2024. Association for Computational Linguistics. URL: <https://aclanthology.org/2024.emnlp-main.785/>, doi:10.18653/v1/2024.emnlp-main.785.
- [86] Zekun Xu, Abhinav Aggarwal, Oluwaseyi Feyisetan, and Nathanael Teissier. A differentially private text perturbation method using regularized mahalanobis metric. In Oluwaseyi Feyisetan, Sepideh Ghanavati, Shervin Malmasi, and Patricia Thaine, editors, *Proceedings of the Second Workshop on Privacy in NLP*, pages 7–17, Online, November 2020. Association for Computational Linguistics. URL: <https://aclanthology.org/2020.privatenlp-1.2/>, doi:10.18653/v1/2020.privatenlp-1.2.
- [87] Ren Yi, Octavian Suci, Adria Gascon, Sarah Meiklejohn, Eugene Bagdasarian, and Marco Gruteser. Privacy reasoning in ambiguous contexts. *arXiv preprint arXiv:2506.12241*, 2025.
- [88] Da Yu, Saurabh Naik, Arturs Backurs, Sivakanth Gopi, Huseyin Inan, Gautam Kamath, Janardhan (Jana) Kulkaarni, Yin Tat Lee, Andre Manoel, Lukas Wutschitz, Sergey Yekhanin, and Huishuai Zhang. Differentially private fine-tuning of language models. In *ICLR 2022*, April 2022. URL: <https://www.microsoft.com/en-us/research/publication/differentially-private-fine-tuning-of-language-models/>.
- [89] Shuozi Yuan, Liming Chen, Miaomiao Yuan, and Zhao Jin. Mcts-sql: light-weight llms can master the text-to-sql through monte carlo tree search. In *Proceedings*

of the Fortieth AAAI Conference on Artificial Intelligence and Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence and Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI'26/IAAI'26/EAAI'26. AAAI Press, 2026. doi:10.1609/aaai.v40i41.40751.

- [90] Xiang Yue, Minxin Du, Tianhao Wang, Yaliang Li, Huan Sun, and Sherman S. M. Chow. Differential privacy for text analytics via natural text sanitization, 2021. URL: <https://arxiv.org/abs/2106.01221>, arXiv:2106.01221.
- [91] Xiang Yue, Huseyin Inan, Xuechen Li, Girish Kumar, Julia McAnallen, Hoda Shajari, Huan Sun, David Levitan, and Robert Sim. Synthetic text generation with differential privacy: A simple and practical recipe. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1321–1342, Toronto, Canada, July 2023. Association for Computational Linguistics. URL: <https://aclanthology.org/2023.acl-long.74/>, doi:10.18653/v1/2023.acl-long.74.
- [92] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, 2024.
- [93] Juhua Zhang, Zhiliang Tian, Minghang Zhu, Yiping Song, Taishu Sheng, Siyi Yang, Qiunan Du, Xinwang Liu, Minlie Huang, and Dongsheng Li. DYNTEXT: Semantic-aware dynamic text sanitization for privacy-preserving LLM inference. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 20243–20255, Vienna, Austria, July 2025. Association for Computational Linguistics. URL: <https://aclanthology.org/2025.findings-acl.1038/>, doi:10.18653/v1/2025.findings-acl.1038.
- [94] Xiaohu Zhu, Qian Li, Lizhen Cui, and Yongkang Liu. Large language model enhanced text-to-sql generation: A survey. *arXiv preprint arXiv:2410.06011*, 2024.

Table 6: Notation used throughout the paper.

Notation	Meaning
$x, S, \mathcal{P}$	Natural language query; schema; privacy policy.
$D, \tilde{D}$	Private; DP synthetic database.
$Q(x, S), q^*$	Candidate queries; selected query.
Γ	Schema integrity constraints.
$I \sim I'$	Neighboring database instances.
GS_q, LS_q, RS_q	Global, local, residual sensitivity.
q_E, B_E	Residual query; boundary attributes.
$m_i(b)$	Boundary multiplicity at b .
LS_i, T_{E_i}	Local sensitivity; maximizing boundary value.
$\mu_{i,1}^{\text{free}}, \mu_{i,\min}^{\text{free}}, M_i^{\text{unocc}}$	Private boundary multiplicities.
$\mathcal{R}_{\text{priv}}, \mathcal{S}_k$	Private relations; distance vectors.
$V_i^{\text{snd}}, V_i^{\text{rec}}, V_i^{\text{sub}}$	Sender, recipient, subject views.
$V_i^\theta, V_i^{\text{data}}$	Condition and data views.
$o = (R, tid, A)$	Source cell provenance.
$\rho = (s_\rho, r_\rho, \pi_\rho)$	Runtime sender, recipient, purpose.
$\text{Pos}_\rho, \text{Viol}_\rho$	Permitted and prohibited information.
V_ρ^{exact}	Information authorized for exact release.
$\mathcal{A}_c(D)$	Authorized database cells.

A Appendix

A.1 Integrating Contextual Integrity and Differential Privacy (Cntd.)

We discuss our inference rules in detail below:

Boundary multiplicities. Recall from Sec. 2 that RS measures how strongly a tuple in R_i can be amplified through the residual join. For R_i , this amplification is determined by the largest boundary multiplicity, $T_{E_i} = LS_i$. Thus, contextual information can tighten the local sensitivity only if it can remove the boundary value realizing this witness from the neighborhood.

Under contextual DP, values authorized by $\mathcal{A}_c(D)$ are fixed across contextual neighbors. Among the occupied boundary values that remain private, let $\mu_{i,1}^{\text{free}}$ and $\mu_{i,\min}^{\text{free}}$ denote the largest and smallest multiplicities, respectively. Under deletion, $\mu_{i,1}^{\text{free}}$ is the largest private contribution that can still be removed. Under change neighborhood, a tuple may move from one boundary value to another, so $\mu_{i,\min}^{\text{free}}$ captures the smallest contribution it can move to; changing from the original witness can therefore change the answer by at most $T_{E_i} - \mu_{i,\min}^{\text{free}}$. Hence, $\mu_{i,\min}^{\text{free}} \leq \mu_{i,1}^{\text{free}} \leq T_{E_i}$.

For insertions, M_i^{unocc} denotes the largest multiplicity at a boundary value not currently occupied by R_i , since a new tuple may introduce such a value. If referential integrity prevents introducing new boundary values, then $M_i^{\text{unocc}} = 0$. The rules below characterize which of these multiplicities remain

reachable under each neighboring edit.

Deletion. When a selection of tuples are contextually-public, i.e., fixed across contextual neighbors, they cannot be deleted. Any valid deletion must come from the remaining private tuples. The worst case is thus the private tuple with the largest boundary multiplicity, giving $\mu_{i,1}^{\text{free}}$ in SEL-DEL. Intuitively, contextual information removes public tuples from the set of possible deletion witnesses.

Change. Under change neighborhood, tuple existence is fixed, but an existing tuple may change the boundary value through which it joins with the residual query. If the complete boundary projection is public, these join relevant values are fixed, so changing the tuple cannot alter its join multiplicity and the sensitivity contribution is zero (PRJ-CHG). If only part of the boundary information is public, a change can still have a large effect in two ways: it can move a tuple away from the original witness, changing its contribution by at most $T_{E_i} - \mu_{i,\min}^{\text{free}}$, or it can move to a fully private boundary value with multiplicity at most $\mu_{i,1}^{\text{free}}$. Taking the larger of these possibilities gives $\max\{T_{E_i} - \mu_{i,\min}^{\text{free}}, \mu_{i,1}^{\text{free}}\}$ in PRJSEL-CHG and in the model allowing both deletion and change (SEL).

Insertion. Insertion is different because a new tuple is not restricted to boundary values already present in the public data. It may introduce a new boundary value and still realize the ordinary worst case, so INS retains LS_i . A tighter bound becomes possible when B_i is unique: with a unique boundary, an insertion cannot reuse an occupied boundary value, so its effect is bounded by M_i^{unocc} , the largest multiplicity at an unoccupied boundary value. Deletion or change may still affect an existing private tuple, with contribution bounded by $\mu_{i,1}^{\text{free}}$. Since the rule must cover all allowed edits, SEL-UNIQUE gives $\max\{\mu_{i,1}^{\text{free}}, M_i^{\text{unocc}}\}$. Referential integrity can further rule out new boundary values, forcing $M_i^{\text{unocc}} = 0$. Finally, if the entire relation is public, every tuple and value is fixed across contextual neighbors, so PUB gives zero sensitivity. Unlike ordinary residual sensitivity, contextual residual sensitivity is query- and context-dependent: the same query on the same database can have different sensitivity for different queriers or purposes because CI authorizes different information flows.

A.2 Proofs

Theorem 4 (End-to-end differential privacy). *If the synthetic release is $(\epsilon_{\text{syn}}, \delta_{\text{syn}})$ -DP and the final query release is $(\epsilon_{\text{rel}}, \delta_{\text{rel}})$ -DP, then PIMENTO is $(\epsilon_{\text{syn}} + \epsilon_{\text{rel}}, \delta_{\text{syn}} + \delta_{\text{rel}})$ -DP.*

Proof. PIMENTO accesses D only twice. First, it releases $\tilde{D}$ with $(\epsilon_{\text{syn}}, \delta_{\text{syn}})$ -DP. The entire search, including sensitivity evaluation and selection of q^* , operates only on $\tilde{D}$ and is therefore post-processing. Second, PIMENTO evaluates q^* on D using an $(\epsilon_{\text{rel}}, \delta_{\text{rel}})$ -DP release mechanism. Although q^* depends on D , conditioned on $\tilde{D}$ it is fixed. The result therefore follows by adaptive composition [22]. $\square$

Theorem 5 (Equivalence of CI Policy and Compiled Views). *Fix a database D and query $\rho = (s_\rho, r_\rho, \pi_\rho, q_\rho)$. Let $\mathcal{P}^+$ denote the implication-completed CI policy, and assume every norm in $\mathcal{P}^+$ is faithfully compiled into its SQL views.*

Let

$$\llbracket \mathcal{P}^+ \rrbracket_{D,\rho} = \left\{ (v, o) \in q_\rho(D) \text{ such that } \begin{array}{l} (v, o) \text{ is permitted by at least one} \\ \text{applicable positive norm of } \mathcal{P}^+ \text{ and} \\ \text{prohibited by no applicable negative} \\ \text{norm of } \mathcal{P}^+ \end{array} \right\}$$

denote the information authorized by the CI policy for ρ . Then the compiled SQL views are equivalent to the CI policy:

$$V_\rho^{\text{exact}} = \text{Pos}_\rho \setminus \text{Viol}_\rho = \llbracket \mathcal{P}^+ \rrbracket_{D,\rho}.$$

Equivalently, for every origin-tagged queried value $(v, o) \in q_\rho(D)$,

$$(v, o) \in V_\rho^{\text{exact}} \iff (v, o) \in \llbracket \mathcal{P}^+ \rrbracket_{D,\rho}.$$

Thus, CI enforcement is both sound and complete: it releases no queried value prohibited by the policy and withholds no queried value authorized by the policy. Consequently, releasing V_ρ^{exact} satisfies contextual integrity with respect to $\mathcal{P}$.

Proof. Fix an origin-tagged queried value $(v, o) \in q_\rho(D)$, where $o = (R, \text{tid}, A)$. We show that (v, o) belongs to V_ρ^{exact} if and only if its flow is authorized by $\mathcal{P}^+$.

First, runtime relevance filtering does not remove any norm that can govern (v, o) . A norm N_i is discarded only if its data view V_i^{data} governs none of the attributes projected by q_ρ . By faithful compilation, V_i^{data} contains exactly the database cells representing information of type τ_i . Hence, if N_i governs (v, o) , then its origin attribute A is represented by V_i^{data} and N_i is retained. Conversely, a discarded norm governs no queried value and therefore cannot affect whether (v, o) belongs to Pos_ρ or Viol_ρ .

Now consider any retained norm N_i . At runtime, PIMENTO evaluates

$$\pi_{q,v,o} \left(\sigma_{p_1=s_\rho} (V_i^{\text{snd}}) \bowtie \sigma_{p_2=r_\rho} (V_i^{\text{rec}}) \bowtie V_i^{\text{sub}} \bowtie V_i^\theta \bowtie V_i^{\text{data}} \right).$$

A tuple (q, v, o) appears in this result if and only if there exists a joint binding of the shared variables (p_1, p_2, q) , with $p_1 = s_\rho$ and $p_2 = r_\rho$, under which all compiled views hold. The natural join therefore implements conjunction of the corresponding conditions.

By faithful compilation, the role views hold exactly when s_ρ, r_ρ , and q instantiate the sender, recipient, and subject roles of N_i . Likewise, V_i^θ holds exactly when the transmission condition θ_i is satisfied and the declared purpose π_ρ matches π_i , while V_i^{data} contains exactly the origin-tagged values of information type τ_i about the bound subject. Therefore,

$$\begin{aligned} (q, v, o) \text{ is returned by the compiled views of } N_i \\ \iff N_i \text{ applies to the flow of } (v, o) \text{ under } \rho. \end{aligned}$$

Taking the union over all applicable positive norms therefore gives exactly

$$\text{Pos}_p = \left\{ (v, o) \in q_p(D) : \begin{array}{l} \text{some positive norm of } \mathcal{P}^+ \\ \text{permits } (v, o) \end{array} \right\},$$

and, by the same argument,

$$\text{Viol}_p = \left\{ (v, o) \in q_p(D) : \begin{array}{l} \text{some negative norm of } \mathcal{P}^+ \\ \text{prohibits } (v, o) \end{array} \right\}.$$

It follows directly that

$$\begin{aligned} (v, o) \in V_p^{\text{exact}} &\iff (v, o) \in \text{Pos}_p \setminus \text{Viol}_p \\ &\iff (v, o) \text{ is permitted by at least one} \\ &\quad \text{applicable positive norm and} \\ &\quad \text{prohibited by no applicable} \\ &\quad \text{negative norm} \\ &\iff (v, o) \in \llbracket \mathcal{P}^+ \rrbracket_{D, p}. \end{aligned}$$

Since this holds for every $(v, o) \in q_p(D)$,

$$V_p^{\text{exact}} = \llbracket \mathcal{P}^+ \rrbracket_{D, p}.$$

Finally, $\mathcal{P}^+$ preserves the permissions of $\mathcal{P}$ and adds only prohibitions induced by information implications. Hence every flow authorized under $\mathcal{P}^+$ is compliant with $\mathcal{P}$, and releasing V_p^{exact} satisfies contextual integrity with respect to $\mathcal{P}$. $\square$

Contextual residual sensitivity. We now prove Theorem 3 of Sec. 4.3: contextual residual sensitivity (CRS_q) is a smooth upper bound on local sensitivity under contextual DP, and it never exceeds ordinary residual sensitivity (RS_q). Throughout, fix a query q and a context c with authorized cells $\mathcal{A}_c(D)$.

Setup. For each private relation R_i , write $\ell_i(I)$ for the bound given by the inference rule of Sec. 4.3 that applies to R_i under $\mathcal{A}_c(D)$, i.e., $\Gamma, \mathcal{A}_c(D) \vdash_I R_i : \tau \Rightarrow \ell_i(I)$. Every contextual neighbor must preserve $\mathcal{A}_c(D)$ (Definition 2), so a relation entirely governed by PUB admits no contextual edit at all: inserting, deleting, or changing any of its tuples would change $\mathcal{A}_c(D)$. We call the remaining relations *contextually editable*,

$$\mathcal{R}_{\text{priv}}^c = \{i \in \mathcal{R}_{\text{priv}} : \ell_i \text{ is not derived by PUB}\},$$

and restrict distance vectors to them, $\mathfrak{S}_k^c = \{\mathbf{s} \in \mathfrak{S}_k : s_j = 0 \text{ for all } j \notin \mathcal{R}_{\text{priv}}^c\}$. The *contextual distance* $d_c(I, I')$ is the fewest edits needed to reach I' from I while preserving $\mathcal{A}_c(D)$ at every intermediate step. Since a contextual edit is in particular an ordinary edit, $d(I, I') \leq d_c(I, I')$, and any instance within contextual distance k of I can only differ from I on

relations in $\mathcal{R}_{\text{priv}}^c$. The *contextual local sensitivity* of q at I is local sensitivity restricted to contextual neighbors,

$$\text{CLS}_{q, \mathcal{A}_c(D)}(I) = \max_{I' : I \sim_c I'} ||q(I)| - |q(I')||.$$

As with ordinary DP, we cannot calibrate noise to $\text{CLS}_{q, \mathcal{A}_c(D)}$ directly, since it can itself change sharply between neighbors. Following Nissim et al. [61], a function $\hat{S}(\cdot)$ is a β -smooth upper bound of local sensitivity under contextual DP if (i) $\hat{S}(I) \geq \text{CLS}_{q, \mathcal{A}_c(D)}(I)$ for every I , and (ii) $\hat{S}(I) \leq e^\beta \hat{S}(I')$ for every $I \sim_c I'$; noise calibrated to any such $\hat{S}$ using their Cauchy or Laplace mechanisms achieves ϵ - or (ϵ, δ) -contextual DP.

Definition 3 (Contextual residual sensitivity). *For $k \geq 0$, the contextual distance- k envelope $\widehat{\text{CLS}}_q^{(k)}$ is defined exactly as the envelope of Eq. (4), but with distance vectors and boundary maxima restricted to $\mathcal{R}_{\text{priv}}^c$:*

$$\widehat{\text{CLS}}_q^{(k)}(I) = \begin{cases} \max_{i \in \mathcal{R}_{\text{priv}}^c} \ell_i(I), & k = 0, \\ \max_{i \in \mathcal{R}_{\text{priv}}^c} \max_{\mathbf{s} \in \mathfrak{S}_k^c} \sum_{A \subseteq E_i \cap \mathcal{R}_{\text{priv}}^c} T_{E_i \setminus A}(I) \prod_{j \in A} s_j, & k \geq 1. \end{cases}$$

The contextual residual sensitivity of q on I is then

$$\text{CRS}_q(I) = \max_{k \geq 0} e^{-\beta k} \min \{ \text{GS}_q, \widehat{\text{CLS}}_q^{(k)}(I) \}.$$

As with RS_q (Eq. (5)), an unbounded GS_q is treated as ∞ . If $\mathcal{R}_{\text{priv}}^c = \emptyset$, every relation is fully authorized, so there is no contextual neighbor left to protect against: $\text{CLS}_{q, \mathcal{A}_c(D)} \equiv 0$ and $\text{CRS}_{q, \mathcal{A}_c(D)} \equiv 0$, and the exact answer may be released.

We prove Theorem 3 through three lemmas, each a contextual counterpart of a result of Dong and Yi [19]: the inference rules are *sound* (Lemma 1, analogous to their Theorem 4.5), the rules are *tight* against T_{E_i} (Lemma 2), and the envelope $\widehat{\text{CLS}}_q^{(k)}$ is *smooth* across contextual neighbors (Lemma 3, analogous to their Lemma 4.11).

Lemma 1 (Rule soundness). *For every instance I , $\text{CLS}_{q, \mathcal{A}_c(D)}(I) \leq \max_{i \in \mathcal{R}_{\text{priv}}^c} \ell_i(I) = \widehat{\text{CLS}}_q^{(0)}(I)$.*

Proof. Let $I \sim_c I'$, so I' is obtained from I by a single edit τ to some private relation R_i , and this edit preserves $\mathcal{A}_c(D)$. Were ℓ_i derived by PUB, every cell of R_i would be authorized and any edit would change $\mathcal{A}_c(D)$; hence $i \in \mathcal{R}_{\text{priv}}^c$.

Dong and Yi show that a single edit to R_i changes the query answer by exactly the multiplicity of the affected boundary value: if the edit inserts or deletes a tuple with boundary value b , then $||q(I)| - |q(I')|| = m_i(b)$; if it instead changes an existing tuple's boundary value from b to b' (non-boundary attributes do not affect the count), then $||q(I)| - |q(I')|| = |m_i(b) - m_i(b')|$. It remains to bound the multiplicities each rule allows.

Base, Ins. No tuple of R_i is fixed by $\mathcal{A}_c(D)$, so b ranges over the whole domain of B_i , and the change is at most $\max_b m_i(b) = T_{E_i}(I) = \text{LS}_i = \ell_i$.

Sel-Del. Deleting an authorized tuple would remove a cell from $\mathcal{A}_c(D)$, so the deleted tuple must be unauthorized, meaning b is a boundary value occupied by a private tuple. The change is thus at most $\mu_{i,1}^{\text{free}} = \ell_i$.

Prj-Chg. Every boundary cell of R_i is authorized and hence fixed across contextual neighbors, so a change can only touch non-boundary attributes: $b' = b$, the answer is unchanged, matching $\ell_i = 0$.

PrjSel-Chg, Sel. The changed or deleted tuple is unauthorized, so its boundary value b is privately occupied, $\mu_{i,\min}^{\text{free}} \leq m_i(b) \leq \mu_{i,1}^{\text{free}}$; a change may move it to any boundary value b' , with $m_i(b') \leq T_{E_i}$. Whichever of b, b' has larger multiplicity, the change is at most $\max\{T_{E_i} - \mu_{i,\min}^{\text{free}}, \mu_{i,1}^{\text{free}}\} = \ell_i$.

Sel-Unique. Because B_i is unique, an inserted tuple (or the new value of a changed tuple) must occupy a boundary value not already present in R_i , whose multiplicity is at most M_i^{unocc} ; a deleted or changed tuple is instead unauthorized and privately occupied, so its multiplicity is at most $\mu_{i,1}^{\text{free}}$. Either way, the change is at most $\max\{\mu_{i,1}^{\text{free}}, M_i^{\text{unocc}}\} = \ell_i$.

Taking the maximum over $i \in \mathcal{R}_{\text{priv}}^c$ gives $\text{CLS}_{q, \mathcal{A}_c(D)}(I) \leq \max_{i \in \mathcal{R}_{\text{priv}}^c} \ell_i(I) = \widehat{\text{CLS}}_q^{(0)}(I)$. $\square$

Lemma 2 (Rule tightness). *For every rule and every instance I , $\ell_i(I) \leq T_{E_i}(I) = \text{LS}_i$.*

Proof. BASE and INS return LS_i exactly. Every other bound is a maximum of $m_i(b)$ over some subset of $\text{dom}(B_i)$, whereas $\text{LS}_i = T_{E_i}(I)$ maximizes over the entire domain (Eq. (1)), and a maximum over a subset can only be smaller. Concretely, $\mu_{i,1}^{\text{free}} \leq \text{LS}_i$ and $M_i^{\text{unocc}} \leq \text{LS}_i$ cover SEL-DEL and SEL-UNIQUE; since $\mu_{i,\min}^{\text{free}} \geq 0$, $T_{E_i} - \mu_{i,\min}^{\text{free}} \leq T_{E_i} = \text{LS}_i$ covers PRJSEL-CHG and SEL; and $0 \leq \text{LS}_i$ covers PRJ-CHG and PUB. $\square$

Lemma 3 (Smoothness). *For every $k \geq 0$ and every pair $I \sim_c I'$, $\widehat{\text{CLS}}_q^{(k)}(I) \leq \widehat{\text{CLS}}_q^{(k+1)}(I')$.*

Proof. We use two properties of T_E established by Dong and Yi [19]. First, $T_E(I)$ depends only on the relations indexed by E and is therefore invariant under edits to relations outside E . Second, for any $j \in E$, a single edit to R_j changes $T_E(I)$ by at most $T_{E \setminus \{j\}}(I)$. These properties allow the residual-sensitivity envelope to account for neighboring databases by incrementing the corresponding coordinates of its distance vector. The same argument applies when the ground set is restricted to $\mathcal{R}_{\text{priv}}^c$: only coordinates corresponding to editable relations can be incremented, and Lemma 1 establishes that every such relation belongs to $\mathcal{R}_{\text{priv}}^c$. Thus, the residual-sensitivity construction carries over with both its ground set and distance vector restricted to $\mathcal{R}_{\text{priv}}^c$.

Let $I \sim_c I'$ differ in relation R_j , so $j \in \mathcal{R}_{\text{priv}}^c$, and write $T_{E,s}^c(I) = \sum_{A \subseteq E \cap \mathcal{R}_{\text{priv}}^c} T_{E-A}(I) \prod_{i \in A} s_i$ for the weighted sum defining $\widehat{\text{CLS}}_q^{(k)}$.

Case $k = 0$. Let $i \in \mathcal{R}_{\text{priv}}^c$ attain $\widehat{\text{CLS}}_q^{(0)}(I) = \ell_i(I)$, which is at most $T_{E_i}(I)$ by Lemma 2. If $j = i$, fact (a) gives $T_{E_i}(I) = T_{E_i}(I')$ directly, since $i \notin E_i$. Otherwise $j \in E_i \cap \mathcal{R}_{\text{priv}}^c$, and fact (b) gives $T_{E_i}(I) \leq T_{E_i}(I') + T_{E_i - \{j\}}(I')$. In both cases the right-hand side is a sum of terms of $T_{E_i, \mathbf{e}_j}^c(I')$ for the unit vector $\mathbf{e}_j \in \mathfrak{S}_1^c$, so $\widehat{\text{CLS}}_q^{(0)}(I) \leq \widehat{\text{CLS}}_q^{(1)}(I')$.

Case $k \geq 1$. Let $(i, \mathbf{s})$ attain $\widehat{\text{CLS}}_q^{(k)}(I)$, and set $\mathbf{s}' = \mathbf{s} + \mathbf{e}_j \in \mathfrak{S}_{k+1}^c$. By the smoothness argument above, applied to the ground set $E_i \cap \mathcal{R}_{\text{priv}}^c$, incrementing coordinate j can only increase the sum: $T_{E_i, \mathbf{s}}^c(I) \leq T_{E_i, \mathbf{s}'}^c(I')$. Hence $\widehat{\text{CLS}}_q^{(k)}(I) \leq T_{E_i, \mathbf{s}'}^c(I') \leq \widehat{\text{CLS}}_q^{(k+1)}(I')$. $\square$

Theorem 6 (Contextual residual sensitivity). *For every multi-way join counting query q , instance I , context c , and smoothing parameter $\beta > 0$:*

- (i) $\text{CRS}_q(I) \leq \text{RS}_q(I)$; and
- (ii) $\text{CRS}_q(\cdot)$ is a β -smooth upper bound of local sensitivity under contextual DP.

Consequently, releasing $|q(I)|$ with general Cauchy (resp. Laplace) noise calibrated to $\text{CRS}_q(I)$, with β set exactly as for residual sensitivity, satisfies ε - (resp. (ε, δ) -) contextual differential privacy, and the injected noise never exceeds that of standard residual sensitivity.

Proof. (i) CRS_q never exceeds RS_q . We show $\widehat{\text{CLS}}_q^{(k)}(I) \leq \text{LS}_q^{(k)}(I)$ for every k ; since $\text{CRS}_q(I)$ and $\text{RS}_q(I)$ apply the same monotone transformation $x \mapsto e^{-\beta k} \min\{\text{GS}_q, x\}$ to these envelopes and maximize over the same range of k (Eq. (5)), this gives $\text{CRS}_q(I) \leq \text{RS}_q(I)$.

For $k = 0$, Lemma 2 gives $\widehat{\text{CLS}}_q^{(0)}(I) = \max_{i \in \mathcal{R}_{\text{priv}}^c} \ell_i(I) \leq \max_{i \in \mathcal{R}_{\text{priv}}^c} T_{E_i}(I) = \text{LS}_q(I)$. For $k \geq 1$, the pair $(i, \mathbf{s})$ attaining $\widehat{\text{CLS}}_q^{(k)}(I)$ satisfies $\mathbf{s} \in \mathfrak{S}_k^c \subseteq \mathfrak{S}_k$, and its defining sum ranges only over $A \subseteq E_i \cap \mathcal{R}_{\text{priv}}^c$ — a subset of the nonnegative terms summed by $\text{LS}_q^{(k)}(I)$, which ranges over $A \subseteq E_i \cap \mathcal{R}_{\text{priv}}$ (Eq. (4)). Dropping terms from a sum of nonnegative quantities cannot increase it, so $\widehat{\text{CLS}}_q^{(k)}(I) \leq \text{LS}_q^{(k)}(I)$.

(ii) CRS_q is a smooth upper bound under contextual DP. We check the two defining conditions. For condition (i),

$$\text{CRS}_{q, \mathcal{A}_c(D)}(I) \geq \min\{\text{GS}_q, \widehat{\text{CLS}}_q^{(0)}(I)\} \geq \text{CLS}_{q, \mathcal{A}_c(D)}(I),$$

since $\widehat{\text{CLS}}_q^{(0)}(I) \geq \text{CLS}_{q, \mathcal{A}_c(D)}(I)$ by Lemma 1, and $\text{GS}_q \geq \text{LS}_q(I) \geq \text{CLS}_{q, \mathcal{A}_c(D)}(I)$ because every contextual neighbor is also a standard neighbor.

For condition (ii), fix $I \sim_c I'$ and let k^* attain the maximum defining $\text{CRS}_q(I)$; such a maximizer exists at some finite k^* , exactly as for standard residual sensitivity (Dong and Yi, Lemma 4.12), with the same argument applying verbatim over $\mathcal{R}_{\text{priv}}^c$. By Lemma 3 and then rescaling by e^β ,

$$\begin{aligned} \text{CRS}_q(I) &= e^{-\beta k^*} \min \{ \text{GS}_q, \widehat{\text{CLS}}_q^{(k^*)}(I) \} \\ &\leq e^{-\beta k^*} \min \{ \text{GS}_q, \widehat{\text{CLS}}_q^{(k^*+1)}(I') \} \\ &= e^\beta \cdot e^{-\beta(k^*+1)} \min \{ \text{GS}_q, \widehat{\text{CLS}}_q^{(k^*+1)}(I') \} \\ &\leq e^\beta \text{CRS}_q(I'). \end{aligned}$$

Since CRS_q satisfies both conditions with respect to $\sim_c$, the Cauchy and Laplace mechanisms of [61], calibrated to $\text{CRS}_q(I)$ with the same β used for residual sensitivity, achieve ϵ - and (ϵ, δ) -contextual DP, respectively. Part (i) then implies this noise is never larger than what standard residual sensitivity would require. $\square$

End-to-end privacy. The following proof establishes Corollary 1 of Sec. 4.3.

Proof. Write the end-to-end mechanism as

$$\mathcal{M}(D) = (\tilde{D}, \mathcal{M}_{\text{rel}}(D; q^*)), \quad \tilde{D} = \mathcal{M}_{\text{syn}}(D), \quad q^* = f(\tilde{D}),$$

where f is the query-selection procedure: a function of $\tilde{D}$ and public inputs (the natural-language question and schema) alone, which never reads D .

By hypothesis, $\tilde{D}$ is $(\epsilon_{\text{syn}}, \delta_{\text{syn}})$ -DP. Since $q^* = f(\tilde{D})$ depends only on $\tilde{D}$, the pair $(\tilde{D}, q^*)$ is a post-processing of $\tilde{D}$, and post-processing preserves DP under any neighboring relation (Prop. 2.1 of [22]), in particular the contextual relation $\sim_c$. So $(\tilde{D}, q^*)$ remains $(\epsilon_{\text{syn}}, \delta_{\text{syn}})$ -DP.

By hypothesis, $\mathcal{M}_{\text{rel}}(\cdot; q)$ is $(\epsilon_{\text{rel}}, \delta_{\text{rel}})$ -contextual DP for every query q in the DP-compatible class. Since q^* always lies in this class, this guarantee holds in particular for $q = q^*$, regardless of which query is realized.

Together, these two facts say that $\mathcal{M}$ is an adaptive composition of two mechanisms: $\mathcal{M}_1(D) = (\tilde{D}, q^*)$, which is $(\epsilon_{\text{syn}}, \delta_{\text{syn}})$ -DP, followed by $\mathcal{M}_2(D; z) = \mathcal{M}_{\text{rel}}(D; z)$, which is $(\epsilon_{\text{rel}}, \delta_{\text{rel}})$ -contextual DP for every fixed z . Contextual DP is exactly (ϵ, δ) -DP with respect to the fixed neighboring relation $\sim_c$ (Definition 2), so the ordinary sequential composition theorem for (ϵ, δ) -DP (Thm. 3.16 of [22]) applies with $\sim_c$ in place of $\sim$, giving that $\mathcal{M}$ is $(\epsilon_{\text{syn}} + \epsilon_{\text{rel}}, \delta_{\text{syn}} + \delta_{\text{rel}})$ -contextual DP.

Theorem 3 shows that calibrating $\mathcal{M}_{\text{rel}}$ to CRS_q satisfies this hypothesis for every q in the DP-compatible class, so PIMENTO instantiated this way achieves the guarantee above. Finally, any additional output that depends only on $\tilde{D}$ and public inputs, such as the SQL text of q^* , incurs no additional privacy cost by post-processing. The authorization boundary

$\mathcal{A}_c(D)$ is instead computed from the private database D under the fixed context c . However, by definition of contextual adjacency, $\mathcal{A}_c(D) = \mathcal{A}_c(D')$ for every $D \sim_c D'$. Thus, $\mathcal{A}_c(D)$ is invariant across contextual neighbors and its use in defining $\sim_c$ or calibrating the release incurs no additional contextual-DP loss. $\square$

A.3 Procedures

Alg. 1: PIMENTO: DP-Aware Text2SQL

Input: question x , schema $\mathcal{S}$, private database D , rollout budget N_{rollout} , warm-start size K
Output: selected query q^*

Initialization

1. $\tilde{D} \leftarrow \mathcal{M}_{\text{syn}}(D; \epsilon_{\text{syn}}, \delta_{\text{syn}})$ ▷ One-time private proxy
2. Initialize root v_0 with
 $\{\text{SET_COUNT}, \text{ADD_RELATION}, \text{STOP}\}$ ▷ DP-compatible tree
3. Set $N(v) \leftarrow 0$, $N(v, a) \leftarrow 0$, $Q(v, a) \leftarrow 0$ ▷ Node visits, edge visits, cumulative reward
4. Set $\{C_y\} \leftarrow \emptyset$, $\{t_y\} \leftarrow \emptyset$ ▷ Answer clusters + best RS per cluster
5. Generate K zero-shot queries and insert their action paths ▷ Warm start
6. **For** each warm-start query q
7. $y \leftarrow \text{bucket}(q(\tilde{D}))$; $C_y \leftarrow C_y \cup \{q\}$ ▷ y : answer; C_y : queries with answer y
8. Compute $\text{RS}_q(\tilde{D})$ and set $t_y \leftarrow \min_{q' \in C_y} \text{RS}_{q'}(\tilde{D})$ ▷ Lowest RS found for answer y
9. Evaluate and backpropagate q along its path ▷ Initialize Q and N
10. **EndFor**

Search and Evaluation

11. **For** rollout $i \in [N_{\text{rollout}}]$
12. $\text{ROLLOUT}(v_0, i, \tilde{D})$ ▷ Explore + evaluate + update
13. **EndFor**

Final Selection

14. $y^* \leftarrow \arg \max_y |C_y|$ ▷ Most supported answer
15. $q^* \leftarrow \arg \min_{q \in C_{y^*}} \text{RS}_q(\tilde{D})$ ▷ Lowest-RS formulation
16. **Return** q^*

Alg. 2: PIMENTO: Rollout

Input: root v_0 , rollout i , synthetic database $\tilde{D}$

Search

1. Select from v_0 using the UCT rule until reaching v with $|\text{children}(v)| < k\sqrt{N(v)}$ $\triangleright$ More visited nodes are allowed
more branches
2. Generate DP-compatible child actions $\triangleright$ LLM expansion
3. Exclude relation keys already used by siblings $\triangleright$ Remove
duplicate branches
4. **For** each $a = \text{ADD_RELATION}(r, J, F)$ producing prefix u'
5. $\hat{q}_{u'} \leftarrow \text{STOP}(u')$ $\triangleright$ Complete current prefix
6. **If** $i \geq \alpha_{\text{prune}} N_{\text{rollout}}$, containment holds, and $\text{RS}_{\hat{q}_{u'}}(\tilde{D}) > \max_y \text{t}_y$
7. Prune u' $\triangleright$ Sensitivity-guided pruning
8. **EndIf**
9. **EndFor**
10. Continue until $\text{STOP}()$ produces terminal query q and trajectory τ

Evaluation

11. **If** q is invalid, nonexecutable, or not DP-compatible
12. Backpropagate 0 and **Return**
13. **EndIf**
14. $y \leftarrow \text{bucket}(q(\tilde{D})); C_y \leftarrow C_y \cup \{q\}$ $\triangleright$ Execution cluster
15. Compute $R_{\text{corr}}(q) \leftarrow \frac{|C_y(q)| - 1}{M - 1}$ $\triangleright$ Reward execution
agreement
16. Compute $\text{RS}_q(\tilde{D})$ and cache $(k^*, s^*, \{T_E\})$
17. Compute $W(U)$ from cached $(k^*, s^*, \{T_E\})$ $\triangleright$
Relation-subset sensitivity
18. **For** each relation $r \in V(q)$
19. $\phi_r(q) \leftarrow \sum_{U \subseteq V \setminus \{r\}} \frac{|U|!(|V| - |U| - 1)!}{|V|!} [W(U \cup \{r\}) - W(U)]$ $\triangleright$ Average marginal contribution
20. **EndFor**

Backpropagation

21. **For** each $(v_{t-1}, a_t) \in \tau$
22. $G_t \leftarrow -\lambda_s v(\phi_{r_t})$ if $a_t = \text{ADD_RELATION}(r_t, J_t, F_t)$; otherwise $G_t \leftarrow 0$ $\triangleright$ Sensitivity
credit
23. $Q(v_{t-1}, a_t) \leftarrow Q(v_{t-1}, a_t) + R_{\text{corr}}(q) + G_t$
24. $N(v_{t-1}, a_t) \leftarrow N(v_{t-1}, a_t) + 1$
25. $N(v_{t-1}) \leftarrow N(v_{t-1}) + 1$
26. **EndFor**
27. Update t_y $\triangleright$ Best formulation so far

A.4 Benchmarks

Differential Privacy-Aware Text2SQL Benchmark (DP-D1). Existing Text2SQL benchmarks do not identify the lowest-sensitivity formulation among correct alternatives, despite sensitivity variation being common: 54.5% of DP-supported BIRD questions and 66.6% of Ambrosia questions produce candidates with different sensitivities (App. ??). We construct DP-D1 by identifying alternative join paths from the foreign-key graphs of BIRD, Spider, Spider2-Lite, AmbiQT and EHRSQL, and use Claude 5 Sonnet to generate alternate SQL formulations and retain correct samples. We compute their RS, and define the minimum-sensitivity formulation as q^* . The benchmark contains 3,673 training questions from BIRD (3,063), AmbiQT (486), and EHRSQL (124), and

Table 7: **Ablation of PIMENTO’s search components** on XiYanSQL-32B. Each row disables one component of the Full configuration in isolation, with AlphaSQL included as a reference. Δ is the change in Joint relative to Full.

Method	Acc.	Is-Min	Joint	Δ	Lat.
Full (PIMENTO)	81.8	76.6	75.3	0.0	159.2
w/o warm-start	79.7	68.3	65.2	-10.1	133.0
w/o pruning	81.8	76.3	74.5	-0.8	235.4
w/o sensitivity prior	81.2	74.7	73.8	-1.5	152.9
w/o sensitivity reward	81.0	66.1	64.7	-10.6	178.7
direct RS reward	81.0	66.4	64.6	-10.7	172.1

Policy	Roles	Data	Purposes	Conditions
CCPA	21	31	31	5
COPPA	12	48	28	15
HIPAA	44	7	6	11

Table 8: Ground-truth policy-to-schema mappings.

886 disjoint test questions from BIRD (415), Spider (362), AmbiQT (64), Spider2-Lite (25), and EHRSQL (20). Alternative formulations differ in Residual Sensitivity by a median of $10.5\times$, demonstrating substantial opportunity for reducing DP noise through SQL formulation. **Contextual Integrity-Aware Text2SQL Benchmark (CONTEXT-D1, CONTEXT-D2, CONTEXT-D3).** No existing Text-to-SQL benchmark captures contextual-integrity policies. We therefore construct one by pairing three real policies with domain-matched Text-to-SQL datasets: HIPAA with 56 EHRSQL queries (health-care privacy), CCPA with 75 BIRD Car Retail queries (consumer privacy), and COPPA with 21 BIRD Computer Student Records queries (children’s online privacy), for 152 queries total. We construct the benchmark in three steps.

Policy extraction. We first use an LLM to extract CI norms specifying roles, information, purposes, conditions, and permit/prohibit decisions. Since prior work extensively studies this task [26, 58, 76, 87], we treat extraction as preprocessing.

Policy grounding. We manually annotate ground-truth mappings from policy concepts to their database representations: roles to identity attributes, information types to data attributes, purposes to database values, and conditions to SQL predicates. Concepts absent from the database are marked unmapped. Table 8 summarizes these annotations.

Query grounding. We sample concrete sender and querier identities and purposes from the database and policy to form runtime contexts. For each query-context pair, we use the mappings to identify applicable norms, evaluate their conditions for the queried subjects, and apply their permit/prohibit decisions to obtain the ground-truth authorized output. If no norm applies, release is denied.

This process produces 495,072 query-context pairs, from

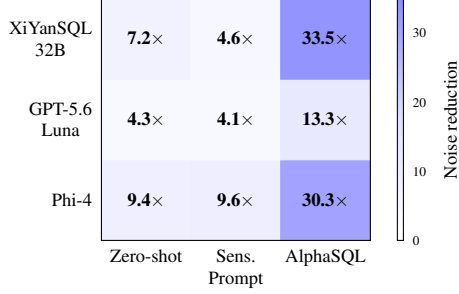


Figure 5: Geometric mean reduction in DP noise relative to each baseline.

which we sample 150 per policy for a 450-instance benchmark. Following prior work [66], we also generate textual narrations of each database. The resulting healthcare, consumer-privacy, and children’s-privacy datasets are denoted CONTEXT-D1, CONTEXT-D2, and CONTEXT-D3, respectively. We use DP-D1 for RQ1 and CONTEXT-D1–CONTEXT-D3 for RQ2–RQ4.

A.5 Evaluation (Cntd.)

A.5.1 RQ1 (Cntd.)

Noise reduction. Fig. 5 (App. A.5) reports the geometric mean reduction in DP noise achieved by PIMENTO. Across models, PIMENTO achieves 4.1–9.6× lower noise than zero-shot and sensitivity prompting, and 13.3–33.5× lower noise than AlphaSQL.

Ablation. We ablate PIMENTO’s four search components on XiYanSQL-32B: warm start, pruning, sensitivity prior, and Shapley-credit reward. We also replace Shapley credit with a coarse residual-sensitivity bonus $\frac{1}{1+\log(\text{RS})}$. Table 7 shows that warm start and Shapley credit have the largest impact, reducing Joint by 10.1 and 10.6 points when removed. Pruning reduces latency by 32.4%. Replacing Shapley credit with the coarse RS bonus reduces Joint by 10.7 points, performing even worse than removing the sensitivity reward entirely.

Cost. Table 9 reports the token cost of PIMENTO vs AlphaSQL. PIMENTO is more efficient than AlphaSQL, using 2.6–5.1× fewer tokens across models.

Table 9: **Inference cost of search-based Text2SQL methods.**

We report the average number of model calls and total tokens consumed per question.

Model	Method	Calls / Q.	Tokens / Q.
GPT-5.6-Luna	AlphaSQL	24.9	40.1K
	PIMENTO	6.6	11.2K
XiYanSQL-32B	AlphaSQL	24.9	19.6K
	PIMENTO	6.6	3.8K
Phi-4	AlphaSQL	24.9	11.4K
	PIMENTO	6.6	4.4K